SE(3)-WAM: Role-Aware 4D World–Action Modeling for Dynamic Manipulation

Anonymous CVPR submission

Figure placeholder: Fig. 1 teaser — a role-aware 4D view of a moving scene (real robot)

Layout: full width, two parts. **(a) Left, ∼62%:** one real-robot frame of the *linear-motion grasp* at the instant a chunk is issued. Overlays in the paper's semantic colors: blue = the target's past SE(3) poses from the causal tracker (small triads at $-350 \ldots 0$ ms, fading with age); orange = the target's SE(3) forecast at the execution times of the chunk, taken from the training-time heads evaluated offline; light orange arrows = sparse 3D flow on the context on the 14×14 grid only; purple = the commanded gripper path meeting the target at τ^{a}. A gray ghost marks the pose observed at capture, with a dashed red arrow showing where a reactive policy would close. A thin timeline strip under the image: capture $\tau_t \rightarrow$ latency $\ell_t \rightarrow$ execution τ^{a}. **(b) Right, ∼38%:** top: "Image-space WAM: future frames or latents at uniform fidelity" (filmstrip icon); middle: "SE(3)-WAM: SE(3) for the target, sparse 3D flow for the context, training only" (triads + sparse arrows); bottom **(required)**: SR vs. added latency (0–200 ms) for SE(3)-WAM and the strongest baseline (data: the latency sweep of Appendix E.2). **Rules:** real photo kept unretouched; overlays drawn from logged tracker/forecast/command data of that trial; caption states the trial's outcome.

Figure 1. **A role-aware 4D view of a moving scene.** Between image capture and the first command, a part on the conveyor moves farther than the grasp tolerates. SE(3)-WAM encodes the target's metric motion (blue) and is trained to forecast the target's SE(3) pose precisely and the surrounding scene as a sparse 3D flow (orange), a forecast it skips at deployment, so that its commands meet the target when they execute (purple). Image-space world–action models forecast appearance at uniform fidelity instead. Right: success against added latency on Dynamic-14.

Abstract

Manipulating moving objects requires actions that fit the scene when they execute, not when the image was captured, yet every millisecond spent inferring the scene's motion lets the target move further. World–action models forecast future images or latents at uniform fidelity, spending time on appearance while leaving the target's metric pose implicit; target-centric predictors are fast but ignore the surroundings that constrain the motion. We observe that the parts of a dynamic scene need different precision: the target's pose decides contact to within millimeters, whereas the rest of the scene matters only through the clearance it leaves. We present SE(3)-WAM, a world–action model built on this role-aware 4D structure. It encodes the target's motion as a metric SE(3) pose history and is trained to forecast the target's future SE(3) trajectory precisely and the rest of the scene coarsely as a sparse 3D flow. The forecasts serve only as training supervision: they are never fed back to the actions and are skipped at deployment, so inference needs neither past frames nor a generated future and stays fast enough for real-time control. On Dynamic-14, a simulated benchmark in which the scene keeps moving during inference, SE(3)-WAM achieves [XX.X]% success, [XX.X] points above the strongest baseline, and succeeds in [XX]% of trials on six real-robot tasks with diverse target motion.

1. Introduction

Learning-based manipulation has advanced rapidly, from diffusion policies to vision–language–action (VLA) and world–action models (WAMs) [9, 16, 37, 56, 75, 84, 106], yet most of this progress assumes that objects stay still unless the robot moves them. In factories, kitchens, and human–robot handovers, objects keep moving while the robot perceives, computes, and acts. This poses two coupled difficulties. A single image does not reveal how the target is moving, and by the time an action executes, the target is no longer where it was seen: a part on a conveyor at 0.1 m/s travels 15 mm during a 150 ms delay, more than a parallel-jaw grasp typically tolerates (Fig. 1).

Recent work equips policies with a memory of past motion and a forecast of the future [19, 49, 67, 94]. Both cost the resource that dynamic manipulation lacks most, time: richer histories and denser forecasts add computation, and the added latency enlarges the very error the forecast is meant to remove. Existing designs therefore trade information against speed. Video-based WAMs forecast whole future frames or latents at uniform fidelity [49, 75, 84], and methods that restrict prediction to dynamic regions or salient tokens still predict in image space [67, 94]. In both, most capacity goes to appearance, and the quantity that decides contact, the target's metric pose when the action executes, stays implicit. Target-centric predictors sit at the other end

[19, 28, 50]: they are fast but blind to surroundings that can block or redirect the motion. None of them provides knowledge of the moving scene that is precise where contact depends on it, coarse elsewhere, and cheap to use at execution time.

Our key observation is that a dynamic policy needs to know very little precisely. Contact depends on the target's pose at the moment the action executes, to within millimeters, whereas the rest of the scene matters only through the clearance it leaves. Nor does this knowledge have to be computed at test time: a forecast that supervises training shapes what the policy represents and can be dropped at deployment. SE(3)-WAM builds on both ideas (Fig. 2). It sees the current image together with short metric SE(3) histories of the target and the robot, which convey motion directly instead of a stack of past frames. A world stream is trained to forecast, in one metric 4D space, the target's future SE(3) trajectory precisely and the rest of the scene as a sparse 3D flow, and an action stream reads the world stream's summary and replans as often as its latency permits. The forecasts never reach the actions and their heads are skipped at deployment, so SE(3)-WAM gains the precision of a metric forecast at a latency close to that of a reactive policy.

We evaluate SE(3)-WAM on Dynamic-14, a simulated benchmark of fourteen tasks in which observation, scene motion, and action execution share one physical clock, so the scene keeps moving while a policy computes. SE(3)-WAM achieves state-of-the-art success of [XX.X]%, [XX.X] points above the strongest baseline, at a p95 latency of [XX] ms, and succeeds in [XX]% of trials on six real-robot tasks, against [XX]% for the strongest baseline. Four final-scale controls isolate the contributions of target-motion history, future supervision, and its two geometric components.

Our contributions are threefold:

- We propose role-aware 4D modeling for dynamic manipulation. Tracing how latency and estimation errors reach the contact shows that only the target needs millimeter precision, so we represent the target precisely by its metric SE(3) pose history and future trajectory, and the rest of the scene coarsely by a sparse 3D flow.
- We present SE(3)-WAM, a world–action model that uses these forecasts only as training supervision for its action stream and skips them at deployment.
- We build Dynamic-14, in which the scene keeps moving during inference, and show state-of-the-art results on it and on six real-robot tasks.

2. Related Work

Dynamic manipulation. Acting on a moving target requires compensating for the time between sensing and acting, a problem as old as feedback control with dead time [64] and predictive tracking [4]. Classical catching and grasping systems estimate the target's trajectory with Kalman-type filters [5], servo on visual features [32], and plan an interception [2, 6, 27], later combined with grasp and reachability planning [1, 52]. These pipelines are precise but engineered for each object and task. Learned visuomotor policies [16, 98] and vision–language–action models (VLAs) [9, 11, 37, 55, 56, 106] generalize across objects and instructions, but they are trained and evaluated in quasi-static scenes, where the world waits while the model computes. Recent work brings learned policies to moving scenes. DOMINO provides a dynamic benchmark together with PUMA, which reads historical optical flow and is supervised with future object features [19]. DynamicManip synthesizes dynamic demonstrations from a single static one [41], DynamicWAM conditions a WAM on flow and kinematic descriptors of the past [49], and DynaWM refines a base VLA's action chunk for moving objects [14]. Other methods forecast salient visual tokens [67], a future image [95], a compact future code with the time to the next gripper event [50], or a 3D Gaussian velocity field that is optimized online [90], and feed the forecast to the policy at test time. Execution schemes reduce or hide the latency instead. DynamicVLA and StreamingVLA overlap inference with execution [60, 81], real-time chunking and bidirectional decoding keep consecutive chunks consistent [10, 46], asynchronous inference decouples prediction from control [62], VLASH rolls the robot state forward to the execution time [68], and Chronos Policy encodes observation and execution timestamps [72]. These schemes change when actions are computed and executed, whereas SE(3)-WAM changes what the policy represents, so the two are orthogonal. Closest to ours, PUMA also uses future prediction only as a training signal. It forecasts mask-pooled image features of the target from 2D flow, whereas SE(3)-WAM forecasts the target's metric SE(3) trajectory and a sparse 3D flow of its surroundings.

World models and world–action models. World models learn how observations evolve, and they have been used for planning with visual foresight [21], for reinforcement learning in a learned latent space [24, 25], and for planning on pretrained visual features [3, 102]. In manipulation, predicted frames or subgoal images have been converted into actions [8, 17, 96], and video prediction has served as pretraining [79], as a predictive objective for inverse dynamics [70], and as a source of predictive features [29]. World–action models (WAMs) generate future video and actions jointly in one network [40, 75, 84, 103]. Recent studies compare how the two streams should exchange information [75, 86] and how long and how compressed their context should be [31, 47]. Because video is expensive and dominated by appearance, a growing body of work changes what is forecast: dynamic regions, depth, and semantic features

[94], flow, geometry, and semantics alongside RGB [88], motion-weighted video [33], object masks [85] or object slots [48], optical flow [15] or motion vectors [42], compact future latents [38, 39], and targets in the policy's own condition or token space [66, 97]. A second question is whether the forecast is needed at test time. Fast-WAM finds that most of the benefit of video modeling remains when test-time imagination is skipped [89], and other methods drop their predictors [19, 66, 97] or auxiliary structure branches [88] after training. SE(3)-WAM follows this training-only use of the forecast but changes its target. Instead of pixels or latents at a uniform resolution, it forecasts a metric 4D state whose precision follows each entity's role: a full SE(3) trajectory for the target and a sparse 3D flow for the rest of the scene. Its action stream reads observation-derived summary tokens shaped by these forecasts, never the forecasts themselves, so the forecast heads are skipped at deployment.

Structured 3D and 4D representations. Metric 3D structure makes manipulation policies more precise and sample-efficient. Voxel, multi-view, and point-cloud policies act on scene geometry [22, 35, 61, 91], object-centric point clouds improve generalization [105], and 3D and 4D VLAs bring 3D and spatiotemporal inputs to large models [93, 99]. Motion has been exposed to policies through 2D point tracks [7, 77, 82, 100], dense pixel motion [54, 57, 58], and 3D flow [23, 87, 101], either as inputs or as intermediate predictions that are then turned into actions. Object-level representations include keypoints [51], object SE(3) trajectories [28, 65, 78], and separate encodings of the robot, the task object, and obstacles [18]. Rigid-body structure has long been used to learn scene dynamics, from SE(3) decompositions of point clouds [12, 13] and 3D dynamic scene memories [83] to scene flow estimation [45, 69], and PointWorld scales action-conditioned 3D point flow to in-the-wild robot data [30]. Point trackers, pose estimators, and feed-forward reconstruction models [34, 73, 76, 80, 92] now make such labels cheap to extract from video. These representations usually apply one granularity to the whole scene, either dense geometry and flow everywhere or poses for every object. SE(3)-WAM assigns granularity by role. The target, where contact depends on millimeters, is described by its SE(3) pose history and forecast, and the rest of the scene, which matters through the clearance it leaves, by a sparse 3D flow. For real robots, we pretrain on DROID with the 3D point tracks that PointWorld extracted from it [30].

3. SE(3)-WAM

SE(3)-WAM (Fig. 2) is a transformer with a world stream and an action stream that share attention. The world stream encodes the current observation and the recent motion and is trained to forecast a role-aware 4D scene. The action stream generates action chunks by conditional flow matching [43] and reads the world only through a few summary tokens. We first state the setting that motivates this design.

3.1. Acting on a Moving Target

Execution times. At query t, a policy receives an image, a depth map, and the robot state captured at time τ_t, together with histories of earlier states, and outputs a chunk $A_t = (\mathbf{a}_{t,0}, \dots, \mathbf{a}_{t,H-1})$ whose j-th command executes at

$$\tau^{\mathrm{a}}_{t,j} = \tau_t + \ell_t + j\Delta_a, \tag{1}$$

where ℓ_t is the latency from capture to the first execution (preprocessing, inference, and actuation) and Δ_a is the control period. A command must fit the scene at $\tau^{\mathrm{a}}_{t,j}$ rather than at τ_t (Fig. 3), and the lead $\Delta_{t,j} = \tau^{\mathrm{a}}_{t,j} - \tau_t$ grows with every millisecond of inference.

Target and context. Let $W_t(\mathbf{x}, \tau) \in \mathbb{R}^3$ be the displacement between τ_t and τ of the scene point at $\mathbf{x}$, in the robot base frame. The *target* $\mathcal{O}$ is the rigid body whose pose error enters a contact or containment tolerance ε, e.g., an object to grasp or a receptacle to place into. The *context* $\mathcal{C}$ contains the bodies that only need to be avoided, such as a belt or an occluder, with clearance m. On the target, the motion field is set by its pose $T(\tau) = (R(\tau), \mathbf{p}(\tau)) \in \mathrm{SE}(3)$:

$$W_t(\mathbf{x}, \tau) = T(\tau)\, T(\tau_t)^{-1}\mathbf{x} - \mathbf{x}, \qquad \mathbf{x} \in \mathcal{O}. \tag{2}$$

Where the error comes from. Let a policy act, explicitly or implicitly, on an estimate $\hat{T}$ of the target pose at τ^{a}. For a contact point at lever arm r and a rotation error θ, the contact error is at most $\|\hat{\mathbf{p}}(\tau^{\mathrm{a}}) - \mathbf{p}(\tau^{\mathrm{a}})\| + 2r\sin(\theta/2)$. Writing the motion over the lead Δ as constant velocity $\mathbf{v}$ plus a residual $\boldsymbol{\rho}$ (acceleration, rolling, impacts) gives

$$\|\hat{\mathbf{p}}(\tau^{\mathrm{a}}) - \mathbf{p}(\tau^{\mathrm{a}})\| \le e_{\mathrm{loc}} + \hat{\Delta}\, e_v + \|\mathbf{v}\|\, e_\tau + e_{\mathrm{dyn}}, \tag{3}$$

the errors of the current position, of the velocity times the lead, of the timing times the speed, and of the residual motion (Appendix G). For the context, a collision requires a prediction error of at least m somewhere along the commanded path. Two conclusions follow. First, the precision that matters is concentrated on the target: a millimeter-level estimate of its pose and motion, including rotation through the lever arm r, whereas the context can be predicted far more coarsely. Second, latency enters the budget twice, through the lead Δ that multiplies every velocity error and through the motion that the residual accumulates, so information that costs inference time pays for itself only if it removes more error than the extra latency adds. These conclusions shape the three parts of SE(3)-WAM: what it remembers (Sec. 3.2), what it learns to forecast (Sec. 3.3), and how its actions use this forecast without paying for it at deployment (Sec. 3.4).

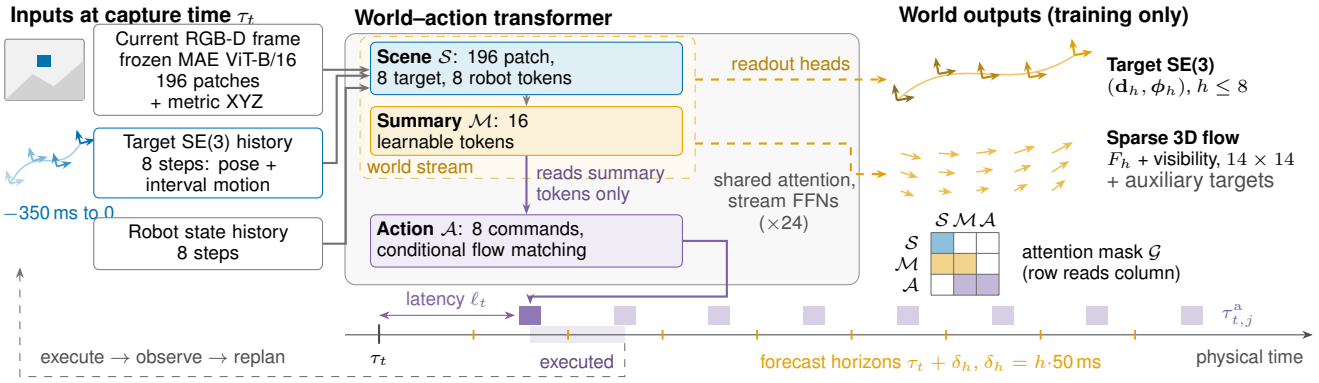


Figure 2. **SE(3)-WAM.** The world stream holds 212 scene tokens, the current image patches from a frozen encoder together with short histories of the target's metric SE(3) pose and of the robot state, and 16 summary tokens. During training, light heads read the world stream out into a role-aware 4D forecast at physical horizons $\tau_t + \delta_h$: the target's SE(3) trajectory and a sparse 3D flow with visibility on a 14×14 grid (dashed: training only). The action stream generates a chunk of 8 commands by flow matching. The attention mask $\mathcal{G}$ lets scene tokens read the scene, summary tokens read the scene and summaries, and actions read summaries and actions, so the forecasts never reach the actions. The policy observes and replans at the first control boundary at or after each query ends.

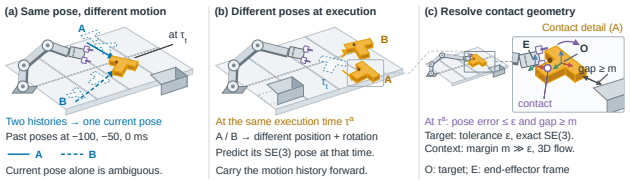


Figure 3. **Acting on a moving target** (schematic). (a) Two histories reach the same pose at capture time τ_t but imply different motions. (b) At the execution time τ^{a}, they lead to different target poses. (c) A contact requires the end-effector frame E to match the target O within ε, whereas the context matters only through the gap it leaves, $m \gg \varepsilon$.

3.2. Inputs: The Current Frame and Motion Histories

SE(3)-WAM sees only one image. A frozen MAE ViT-B/16 [26] encodes the current RGB frame into 196 patch tokens, and the depth map gives each patch the metric 3D position of its center point in the base frame. The motion comes from two short histories over the last 8 control steps (350 ms at 20 Hz). The target history is the current target pose in the base frame together with the seven interval motions between consecutive samples: the translation, the relative rotation, the time interval, and the finite-difference velocity and acceleration, in physical units (Appendix A.1). It forms 8 target-state tokens. The robot history forms 8 end-effector state tokens. Together with the patch tokens, these are the 212 scene tokens. On the real robot, a marker-free causal tracker estimates the target state from the RGB-D stream; each estimate is emitted once with its capture time, and later frames do not revise it. The real-ball protocol retains translation and masks target angular features (Appendix A.2). In simulation, we read the target pose from the simulator, which separates the policy from tracking errors. We evaluate synthetic pose noise separately; actual tracker calibration and closed-loop tracking remain pending (Appendix F.2). The model receives no past images, so its visual encoding cost does not grow with the history.

3.3. Role-Aware 4D Forecast

The world stream is trained to forecast W_t at horizons $\tau_t + \delta_h$, with $\delta_h = h \cdot 50$ ms and $h = 1, \ldots, 8$, which span the execution times of a whole chunk (400 ms). We parameterize the target through Eq. (2). For each horizon,

$$\mathbf{d}_h = \mathbf{p}(\tau_t + \delta_h) - \mathbf{p}(\tau_t), \quad \phi_h = \mathrm{Log}\big(R(\tau_t + \delta_h) R(\tau_t)^\top\big)^\vee, \tag{4}$$

and a pose head regresses $(\mathbf{d}_h, \phi_h)$ with a squared translation error plus λ_R times the squared geodesic rotation error ($\mathcal{L}_P$). The rest of the scene is a sparse 3D flow: for the center point of each of the 196 patches on the 14×14 grid, about [X] mm apart in the workspace, a flow head regresses its base-frame displacements $F_h \in \mathbb{R}^3$ and its visibility at every horizon ($\mathcal{L}_F$). The grid is deliberately coarse, since the context is tolerated up to its clearance, and the target's precision comes from its pose rather than from its points. We also use auxiliary appearance targets ($\mathcal{L}_{\mathrm{aux}}$) only during training, skipping their heads at deployment (Appendix A.3). All heads are light readouts of the final hidden states of the world stream; motion labels are cumulative from the current state, and the losses use valid labels only.

3.4. Actions and the Shared Backbone

The world stream holds the 212 scene tokens and 16 learnable summary tokens. The action stream holds 8 tokens, one per command, each embedding a noisy command and the

flow time. A chunk spans 8 control periods of $\Delta_a = 50$ ms, and each command is an end-effector translation and rotation plus a gripper command. The attention mask $\mathcal{G}$ lets scene tokens attend only to the scene, summary tokens to the scene and summaries, and action tokens to summaries and actions. The actions therefore read the world only through the summary tokens, whose content the forecast losses shape during training, while the forecasts themselves never enter the action stream. Both streams share the attention projections of each layer and have separate feed-forward networks, so they exchange information in attention but keep stream-specific capacity. For a normalized chunk $\mathbf{a}_1$, noise $\mathbf{a}_0 \sim \mathcal{N}(0, I)$, and $u \sim \mathcal{U}[0, 1]$, the action loss is

$$\mathcal{L}_A = \mathbb{E}\ \mathrm{MSE}_{\mathrm{valid}}\big(v_\psi(\mathbf{a}_u, u \mid \mathcal{S}_t, \mathcal{G}), \mathbf{a}_1 - \mathbf{a}_0\big), \quad (5)$$

with $\mathbf{a}_u = (1-u)\mathbf{a}_0 + u\mathbf{a}_1$ and the scene tokens $\mathcal{S}_t$ of query t. The total loss is $\mathcal{L} = \mathcal{L}_A + \lambda_F\mathcal{L}_F + \lambda_P\mathcal{L}_P + \lambda_{\mathrm{aux}}\mathcal{L}_{\mathrm{aux}}$. The 4-layer screening study (Appendix F.4) compares alternative couplings: R0 disables the forecast losses; R2 adds world queries whose hidden states the actions read at every layer; R3 lets these queries read the actions, and R4 opens both directions; and R5 encodes the decoded forecast as geometric tokens for the actions (Appendix A.5).

Deployment. At test time, the forecast heads are skipped. Because world tokens never attend to actions, the world stream runs once per query, and the action stream integrates 5 Euler steps on its cached keys and values. The new chunk starts at the next control-period boundary, where the policy observes again and starts the next query, so a query that fits in one control period yields a fresh command for every period (Appendix A.7). The p95 time-to-action is [XX] ms on an RTX 5090.

Implementation. The transformer has 24 layers of width 1024 with 16 heads and feed-forward width 4096, with 504.0M backbone parameters and 523.8M trainable parameters in total. The MAE ViT-B/16 encoder (85.8M) and the DINOv3 teacher (303M) stay frozen. The Dynamic-14 recipe uses all windows of 800 demonstrations per task for 20 epochs (51,740 updates at a global batch size of 512) with AdamW and a cosine schedule, with $\lambda_F = \lambda_P = 1$, $\lambda_R = \lambda_{\mathrm{aux}} = 0.1$. Final training hardware and duration remain [N] RTX PRO 6000 GPUs and [X] hours per seed (Appendix A.6).

3.5. Pretraining for Real Robots

Real robot data do not provide object poses, so the real-robot base learns the parts of the forecast that DROID [36] supports. It has the same architecture with 48 layers (1.02B trainable parameters) and the same inputs except the target history. Its targets are the actions, the sparse 3D flow, and the future DINOv3 features. The flow labels come from the 3D point tracks and the optimized camera calibrations that PointWorld released for DROID [30], resampled to one track per image patch and to DROID's native timing. We configure training on 538,740 windows from 37,874 training episodes for 21,060 updates on 64 GPUs; pretraining is ongoing (Appendix B). The real-robot recipe initializes from this base, adds the target-state tokens and the pose head, and fine-tunes on task demonstrations with tracked poses. The simulation model is trained from the generic initialization, a frozen MAE encoder with a randomly initialized transformer.

4. Experiments

We compare SE(3)-WAM with reactive, dynamic, and world–action policies in simulation and on a real robot (Secs. 4.2 and 4.3) and then ablate its target history and its future supervision (Sec. 4.4).

4.1. Setup

Dynamic-14. We built fourteen tasks in MuJoCo/robosuite [71, 104] in five families: grasping a moving target (harmonic, conveyor, translating, rotating, and occluded motion), placing into a moving receptacle, passing a wand along a rotating wire, capturing rolling or sliding objects, and grasping after an impact (Appendix C, Fig. S3). We vary the added latency (0–200 ms) within the original task motion ranges. Experts with privileged simulator state provide 800 demonstrations per task. We select configurations on development scenes and report the main Dynamic-14 comparisons and final-model ablations on 1,400 held-out scenes (100 per task, new seeds). The final-model ablations average three training seeds; the screening studies state their own seeds and splits in Appendix F. Each baseline is trained once. We give paired 95% intervals from a bootstrap over held-out scenes.

Protocol. Observation, scene motion, and execution share one physical clock with a control period of 50 ms. All methods run under the same executor, and physics advances while a method computes, by its measured latency on one RTX 5090. A policy is queried again once a command has executed and its previous inference has finished, and until a new chunk arrives, the executor continues with the previous one. The +150 ms setting adds 150 ms of delay for every method, and the paused setting stops physics during inference (Appendix C.3). We report the task-balanced success rate (SR), the contact-time error e_t, the position error at first contact e_p, and the p95 time-to-action.

Baselines. We compare with reactive policies (ACT [98], Diffusion Policy [16], and $\pi_{0.5}$ [56]), dynamic VLAs (DynamicVLA [81] and PUMA [19]), image- or latent-space WAMs (DreamVLA [94] and OpenWAM-α [75]), and a

dynamic WAM (DynamicWAM [49]). Each baseline uses its official code, inputs, and recipe, is trained on the same demonstrations, and receives a rendered wrist camera if its recipe uses one (Appendix D). SE(3)-WAM uses only the agent view, but it also uses depth and the target pose. To control access to target-motion information, $\pi_{0.5}$ and DynamicWAM also receive our SE(3) history tokens (+SE(3)); their depth inputs still differ from ours. The *CV input* variant is SE(3)-WAM without forecast losses that receives the constant-velocity extrapolation of the target pose as input tokens.

4.2. Main Results

SE(3)-WAM reaches [XX.X]% SR on held-out Dynamic-14 scenes (Table 1), [X.X] points above the strongest baseline, [name of the strongest baseline], and [X.X] points above the strongest baseline that receives the same SE(3) history, [name]. The margin is largest on [rotating targets and on fast conveyor and translating targets], where the target moves farthest during the lead. Its p95 time-to-action of [XX] ms is [X]$\times$ lower than that of the image-space WAMs, whose latency adds to the lead they must bridge. With 150 ms of added delay, SE(3)-WAM keeps [XX.X]%, whereas reactive policies drop to [XX.X]% and image-space WAMs to [XX.X]%. With physics paused during inference, the difference to [name of the strongest paused baseline] is [$\pm$X.X] points, which separates the contribution of speed from that of the representation. The CV-input variant trails SE(3)-WAM by [X.X] points. We interpret this difference together with the no-future control, which has neither forecast losses nor extrapolated input tokens. Results on DOMINO [19] are in Appendix D.2.

4.3. Real-Robot Experiments

We evaluate six tasks on a Franka arm ([model]) with [gripper model] and a fixed RealSense D435 RGB-D camera (Fig. S4, Table S9), comparing SE(3)-WAM with $\pi_{0.5}$, DynamicWAM, and Diffusion Policy [16]: conveyor grasping (T1), placement into a receptacle on the conveyor (T2), turntable grasping (T3), grasping a moving toy car (T4), intercepting a ball after a collision (T5), and intercepting an object that rolls or slides down a ramp (T6). Together they cover linear, circular, free planar, post-impact, and gravity-accelerated target motion. Each method is trained or fine-tuned per task on the same 100 teleoperated demonstrations, with 20 separate validation demonstrations, and runs 20 trials per task with shared trial keys. SE(3)-WAM tracks the target with [tracker] and runs on one RTX 5090 with a p95 time-to-action of [XX] ms. It succeeds in [XX.X]% of all trials, against [XX.X]% for [the strongest baseline] (Table 2). [Where its advantage is largest, and its dominant failure mode] (Fig. 4).

Figure placeholder: Fig. 4 — real-robot trials

Rows: a success of SE(3)-WAM; the strongest baseline on the same trial key; a failure of SE(3)-WAM. **Columns:** the query frame, the first execution time τ^{a}, and contact. **Overlays:** blue tracked past poses; purple commanded gripper path. Real photos, unretouched; overlays from logged data of each trial.

Figure 4. **Real-robot trials** of SE(3)-WAM and [the strongest baseline] on the same trial key (top two rows) and a failure of SE(3)-WAM (bottom). [One-sentence description of the selected trials.]

4.4. Ablations: What Does a Dynamic Policy Need?

Four core controls use the final 24-layer model and recipe, with three training seeds per control and the same data and training budget (Table 3). They test target history, future supervision, scene-flow supervision, and dedicated target-pose supervision. Each is evaluated under the real-time protocol at 0 and 150 ms of added delay, reusing the full-model reference from Sec. 4.2. Broader design choices were screened with 4-layer models under the group-specific recipes in Appendix F.

What to remember. The no-history control keeps the current image, current target pose, and eight robot states, and removes past target poses and their velocity and acceleration features. Its SR difference from the full model is [signed X.X] points at 0 ms and [signed X.X] points at 150 ms added delay (Fig. 5a). Visual-history alternatives remain part of the 4-layer screening study; they are not final-scale controls. Pose-noise robustness is evaluated separately with frozen weights (Appendix F.2).

What to predict. The no-future control disables SE(3), scene-flow and visibility, and future depth, DINO, and dynamic-RGB losses, while retaining current-target RGB reconstruction. It measures the effect of future supervision as a whole (Fig. 5b). The other two controls keep all appearance targets: one removes only scene-flow and visibility supervision, and the other removes only dedicated SE(3) supervision. The latter retains the same 14×14 flow across the target, robot, and surroundings, giving every part of the scene the same coarse representation. The former measures scene-flow supervision, not environment information alone. Their signed SR differences are [X.X] and [X.X] points at 0 ms and [X.X] and [X.X] at 150 ms; Fig. 5c breaks these effects down by task family.

Table 1. **Main results.** Held-out SR (%) on Dynamic-14 (1,400 new scenes, task-balanced, real time, three training seeds for our variants and one for baselines) by family, with 150 ms of added delay, and with physics paused during inference, together with the contact-time error e_t (ms), the position error at first contact e_p (mm), and the p95 time-to-action on one RTX 5090. Inputs: R, RGB and robot state. D, depth. P, target SE(3) history. Best in bold. Intervals in Appendix E.

		Dynamic-14 SR, real time unless noted										
Method	Inputs	All	Grasp	Place	Pass	Capture	Impact	+150 ms	Paused	e_t	e_p	p95 (ms)
ACT [98]	R	—	—	—	—	—	—	—	—	—	—	—
Diffusion Policy [16]	R	—	—	—	—	—	—	—	—	—	—	—
$\pi_{0.5}$ [56]	R	—	—	—	—	—	—	—	—	—	—	—
$\pi_{0.5}$ + SE(3)	R+P	—	—	—	—	—	—	—	—	—	—	—
DynamicVLA [81]	R	—	—	—	—	—	—	—	—	—	—	—
PUMA [19]	R	—	—	—	—	—	—	—	—	—	—	—
DreamVLA [94]	R	—	—	—	—	—	—	—	—	—	—	—
OpenWAM-α [75]	R	—	—	—	—	—	—	—	—	—	—	—
DynamicWAM [49]	R	—	—	—	—	—	—	—	—	—	—	—
DynamicWAM + SE(3)	R+P	—	—	—	—	—	—	—	—	—	—	—
SE(3)-WAM, CV input	RD+P	—	—	—	—	—	—	—	—	—	—	—
SE(3)-WAM	RD+P	—	—	—	—	—	—	—	—	—	—	—

Figure placeholder: Fig. 5 — ablations (four panels)

(a) History. SR difference between the full model and no target-motion history at 0 and 150 ms added delay. **(b) Future supervision.** Full model vs. all future losses off, grouped by task family. **(c) Geometry supervision.** Full model vs. no scene flow and no dedicated SE(3), grouped by task family. **(d) Success and latency.** p95 time-to-action vs. SR for the full model and these four controls, at 0 and 150 ms added delay. All panels reuse the same held-out trajectories and three training seeds; show paired 95% bootstrap intervals.

Figure 5. **Core ablations at the final model scale.** (a) Target-motion history. (b) Future supervision. (c) The two geometric supervision components. (d) Their success–latency tradeoff under the real-time protocol. [Summarize the measured effects after the four controls are complete.]

Table 2. **Real-robot results.** Success rate (%) on six tasks: T1 conveyor grasping, T2 moving-receptacle placement, T3 turntable grasping, T4 moving-car grasping, T5 post-collision interception, and T6 ramp interception (20 trials per task and method). All pools the trials; intervals in Appendix D.3.

Method	T1	T2	T3	T4	T5	T6	All
$\pi_{0.5}$	—	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—	—
Diffusion Policy	—	—	—	—	—	—	—
SE(3)-WAM	—	—	—	—	—	—	—

Table 3. **Core ablations** of the final 24-layer model on held-out Dynamic-14 scenes (three training seeds). Each row changes one component of SE(3)-WAM (*). SR, SR_{150}: success (%) without and with 150 ms of added delay. e_t: contact-time error (ms). e_p: position error at first contact (mm). p95: time-to-action (ms).

Variant	SR	SR_{150}	e_t	e_p	p95
SE(3)-WAM *	—	—	—	—	—
No target history	—	—	—	—	—
No future supervision	—	—	—	—	—
No scene-flow supervision	—	—	—	—	—
No dedicated SE(3)	—	—	—	—	—

How to use the forecast. The final model uses the forecast only as training supervision. The 4-layer coupling study found no consistent gain from feeding predictions to actions across its three seeds (Appendix F.4); it does not establish equivalence at the final scale. On the final model, enabling unused forecast heads checks action equality for fixed inputs and noise and measures their extra cost. Under the real-time protocol, this extra latency can change the closed-loop trajec-

Figure placeholder: Fig. 6 — forecasts at the execution time

Rows: a rolling capture (median forecast error) and a post-impact grasp that fails. **Columns:** the query frame at τ_t, the frame at τ^a, and a 3D inset in the base frame. **Overlays:** blue past poses, orange forecast triads and coarse flow arrows from the training-time heads, gray ground truth at the same times, purple commanded path. Full set in Fig. S5.

Figure 6. **Forecasts at the execution time** on held-out Dynamic-14 episodes, read from the training-time heads. Top: the SE(3) forecast follows a rolling target that extrapolation overshoots. Bottom: before an impact, the regressed forecast averages the possible outcomes, and the grasp misses.

tory. Figure 5d reports measured success and latency for the four trained controls, without a final-scale R5 comparison.

5. Conclusion

We presented SE(3)-WAM, a world–action model for manipulating moving objects that spends precision where contact depends on it and time only where it pays off. It remembers the target's metric SE(3) motion instead of past frames, learns to forecast the target's SE(3) trajectory precisely and the rest of the scene as a sparse 3D flow, and lets its actions read a representation shaped by these forecasts without generating them at test time. On Dynamic-14, where the scene keeps moving while a policy computes, and on six real-robot tasks, it improves on reactive, dynamic, and world–action policies. Four final-scale controls examine the metric motion history, future supervision, and its two geometric components. [State the supported component-level conclusions after the core ablations.] The design uses forecasts to train the representation while keeping their heads outside the deployed action path.

Limitations. SE(3)-WAM assumes a rigid target that moves on its own until contact, so deformable or articulated targets and post-contact dynamics are outside its scope. It depends on a pose estimate of the target, and tracker failures propagate to the policy; in simulation we read the pose from the simulator. The regressed target forecast cannot represent multimodal futures, e.g., before an impact. Our real-robot base learns from DROID without target poses, so its pose-related parts are learned only during fine-tuning. Finally, our tasks have a single target and slow context motion.

LLM Usage Statement

[To be completed before submission: actual tools used, affected parts of the research and manuscript, and extent of use.]

References

[1] Iretiayo Akinola, Jingxi Xu, Shuran Song, and Peter K. Allen. Dynamic grasping with reachability and motion awareness. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 9422–9429, 2021. 2

[2] Peter K. Allen, Aleksandar Timcenko, Billibon Yoshimi, and Paul Michelman. Automated tracking and grasping of a moving object with a robotic hand-eye system. *IEEE Transactions on Robotics and Automation*, 9(2):152–165, 1993. 2

[3] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025. 2

[4] Ronald Azuma and Gary Bishop. Improving static and dynamic registration in an optical see-through HMD. In *Proceedings of SIGGRAPH*, pages 197–204, 1994. 2

[5] Yaakov Bar-Shalom, X. Rong Li, and Thiagalingam Kirubarajan. *Estimation with Applications to Tracking and Navigation*. Wiley, 2001. 2, 34

[6] Berthold Bäuml, Thomas Wimböck, and Gerd Hirzinger. Kinematically optimal catching a flying ball with a hand-arm-system. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2010. 2

[7] Homanga Bharadhwaj, Roozbeh Mottaghi, Abhinav Gupta, and Shubham Tulsiani. Track2Act: Predicting point tracks from internet videos enables generalizable robot manipulation. In *European Conference on Computer Vision*, 2024. 3

[8] Kevin Black, Mitsuhiko Nakamoto, Pranav Atreya, Homer Walke, Chelsea Finn, Aviral Kumar, and Sergey Levine. Zero-shot robotic manipulation with pre-trained image-editing diffusion models. In *International Conference on Learning Representations*, 2024. 2

[9] Kevin Black, Noah Brown, Danny Driess, et al. π_0: A vision-language-action flow model for general robot control. In *Proceedings of Robotics: Science and Systems*, 2025. 1, 2

[10] Kevin Black, Manuel Y. Galliker, and Sergey Levine. Real-time execution of action chunking flow policies. In *Advances in Neural Information Processing Systems*, 2025. 2

[11] Anthony Brohan, Noah Brown, Justice Carbajal, et al. RT-1: Robotics transformer for real-world control at scale. In *Proceedings of Robotics: Science and Systems*, 2023. 2

[12] Arunkumar Byravan and Dieter Fox. SE3-Nets: Learning rigid body motion using deep neural networks. In *IEEE International Conference on Robotics and Automation*, 2017. 3

[13] Arunkumar Byravan, Felix Leeb, Franziska Meier, and Dieter Fox. SE3-Pose-Nets: Structured deep dynamics models for visuomotor control. In *IEEE International Conference on Robotics and Automation*, 2018. 3

[14] Chongkei Chang and Zhidong Deng. DynaWM: A base-VLA-guided world foundation model for moving-object manipulation. *arXiv preprint arXiv:2607.02604*, 2026. 2

[15] Yixiang Chen, Peiyan Li, Yuan Xu, Qisen Ma, Jiabing Yang, Kai Wang, Jianhua Yang, Dong An, He Guan, Gaoteng Liu, Jianlou Si, Jun Huang, Jing Liu, Nianfeng Liu, Yan Huang, and Liang Wang. FlowWAM: Optical flow as a unified action representation for world action models. *arXiv preprint arXiv:2607.13017*, 2026. 3

[16] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin C. M. Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems*, 2023. 1, 2, 5, 6, 7, 23, 24

[17] Yilun Du et al. Learning universal policies via text-guided video generation. In *Advances in Neural Information Processing Systems*, 2023. 2

[18] David Emukpere, Romain Deffayet, Bingbing Wu, Romain Brégier, Michael Niemaz, Jean-Luc Meunier, Denys Proux, Jean-Michel Renders, and Seungsu Kim. Disentangled object-centric image representation for robotic manipulation. *arXiv preprint arXiv:2503.11565*, 2025. 3

[19] Heng Fang, Shangru Li, Shuhan Wang, Xuanyang Xi, Dingkang Liang, and Xiang Bai. Towards generalizable robotic manipulation in dynamic environments. *arXiv preprint arXiv:2603.15620*, 2026. 1, 2, 3, 5, 6, 7, 23, 24

[20] Gunnar Farnebäck. Two-frame motion estimation based on polynomial expansion. In *Proceedings of the Scandinavian Conference on Image Analysis*, pages 363–370, 2003. 30

[21] Chelsea Finn and Sergey Levine. Deep visual foresight for planning robot motion. In *IEEE International Conference on Robotics and Automation*, 2017. 2

[22] Ankit Goyal, Jie Xu, Yijie Guo, Valts Blukis, Yu-Wei Chao, and Dieter Fox. RVT: Robotic view transformer for 3D object manipulation. In *Conference on Robot Learning*, pages 694–710, 2023. 3

[23] Jun Guo, Xiaojian Ma, Yikai Wang, Min Yang, Huaping Liu, and Qing Li. FlowDreamer: A RGB-D world model with flow-based motion representations for robot manipulation. *arXiv preprint arXiv:2505.10075*, 2025. 3

[24] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640(8059), 2025. 2

[25] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations*, 2024. 2

[26] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16000–16009, 2022. 4, 14, 30

[27] B. Hove and Jean-Jacques E. Slotine. Experiments in robotic catching. In *American Control Conference*, 1991. 2

[28] Cheng-Chun Hsu, Bowen Wen, Jie Xu, Yashraj Narang, Xiaolong Wang, Yuke Zhu, Joydeep Biswas, and Stan Birchfield. SPOT: SE(3) pose trajectory diffusion for object-centric manipulation. In *IEEE International Conference on Robotics and Automation*, 2025. 2, 3

[29] Yucheng Hu, Yanjiang Guo, Pengchao Wang, Xiaoyu Chen, Yen-Jen Wang, Jianke Zhang, Koushil Sreenath, Chaochao Lu, and Jianyu Chen. Video prediction policy: A generalist robot policy with predictive visual representations. In *International Conference on Machine Learning*, 2025. 2

[30] Wenlong Huang, Yu-Wei Chao, Arsalan Mousavian, Ming-Yu Liu, Dieter Fox, Kaichun Mo, and Li Fei-Fei. PointWorld: Scaling 3D world models for in-the-wild robotic manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026. 3, 5, 16

[31] Wei Huang, Bohan Zhang, Chenzhi Liu, Isabella Liu, Shuai Yang, Weian Mao, Luozhou Wang, Yicheng Xiao, Weifeng Lin, Qixin Hu, Bryan Chu, Sifei Liu, Linxi Fan, Xiaojuan Qi, Song Han, and Yukang Chen. Long-WAM: Scaling the context of world-action models. *arXiv preprint arXiv:2610.10528*, 2026. 2

[32] Seth Hutchinson, Gregory D. Hager, and Peter I. Corke. A tutorial on visual servo control. *IEEE Transactions on Robotics and Automation*, 12(5):651–670, 1996. 2

[33] Haoyuan Ji, Lingxiang Fan, Shang Su, Yinqiao Lu, Mengkai Shi, Jun Gao, and Shuo Feng. DC-WAM: Dynamic-centric visual supervision and reasoning for world-action models. *arXiv preprint arXiv:2607.25918*, 2026. 3

[34] Nikita Karaev, Ignacio Rocco, Benjamin Graham, Natalia Neverova, Andrea Vedaldi, and Christian Rupprecht. CoTracker: It is better to track together. In *European Conference on Computer Vision*, 2024. 3

[35] Tsung-Wei Ke, Nikolaos Gkanatsios, and Katerina Fragkiadaki. 3D diffuser actor: Policy diffusion with 3D scene representations. In *Conference on Robot Learning*, 2024. 3

[36] Alexander Khazatsky, Karl Pertsch, Suraj Nair, et al. DROID: A large-scale in-the-wild robot manipulation dataset. In *Proceedings of Robotics: Science and Systems*, 2024. 5, 16

[37] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, et al. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning*, 2024. 1, 2

[38] Fenghao Lei, Zhixiong Huang, Long Yang, Jiabao Chen, Peilin Huang, Han Fu, Zhuo Li, and Xiaoxue Ren. DELEw0.5: Inferring action from future latent state for robotic manipulation. *arXiv preprint arXiv:2608.22067*, 2026. 3

[39] Jiajun Li, Tiecheng Guo, Yifan Ye, Rongyu Zhang, Xiaowei Chi, Qianpu Sun, Ying Li, Yunfan Lou, Jinqiao Wang, Zhihe Lu, Meng Guo, and Shanghang Zhang. Efficient-WAM: A 1B-parameter world-action model with low-cost future imagination. *arXiv preprint arXiv:2606.10040*, 2026. 3

[40] Shuang Li, Yihuai Gao, Dorsa Sadigh, and Shuran Song. Unified video action model. In *Proceedings of Robotics: Science and Systems*, 2025. 2

[41] Haoran Liao, Pengyue Wang, Shuoyu Chen, Kehan Cheng, Xuhang Chen, Yuhao Lin, Mu Lin, Zhizhao Liang, Xiaoyi Fan, Chengyi Xing, Dan Niu, Yi-Lin Wei, and Wei-Shi Zheng. DynamicManip: Enabling dynamic manipulation from a single static demonstration. *arXiv preprint arXiv:2608.01452*, 2026. 2

[42] Minghui Lin, Pengxiang Ding, Shu Wang, Zifeng Zhuang, Yang Liu, Xinyang Tong, Wenxuan Song, Shangke Lyu, Siteng Huang, and Donglin Wang. HiF-VLA: Hindsight, insight and foresight through motion representation for vision-language-action models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026. 3

[43] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023. 3

[44] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, and Lei Zhang. Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. In *European Conference on Computer Vision*, 2024. 14

[45] Xingyu Liu, Charles R. Qi, and Leonidas J. Guibas. FlowNet3D: Learning scene flow in 3D point clouds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019. 3

[46] Yuejiang Liu, Jubayer Ibn Hamid, Annie Xie, Yoonho Lee, Maximilian Du, and Chelsea Finn. Bidirectional decoding: Improving action chunking via guided test-time sampling. In *International Conference on Learning Representations*, 2025. 2

[47] Yushan Liu, Jingjing Fan, Shoujie Li, Yifan Xie, Xiao-Ping Zhang, and Wenbo Ding. PACT-WAM: Predicting actions and visual foresight with compact temporal encoding for robot manipulation. *arXiv preprint arXiv:2602.15882*, 2026. 2

[48] Yushan Liu, Peibo Sun, Shoujie Li, Yifan Xie, Lingfeng Zhang, Xintao Chao, Shiyuan Dong, Fang Chen, Xiao-Ping Zhang, and Wenbo Ding. OA-WAM: Object-addressable world action model for robust robot manipulation. *arXiv preprint arXiv:2605.06481*, 2026. 3

[49] Yunfan Lou, Hewen Gao, Xiyu Zhu, Zhuoran Qiao, Xuan Han, Yifan Yang, Yifan Ye, Boxian Yao, and Zhibo Pang. DynamicWAM: Dual-path motion conditioning for world-action models in dynamic manipulation. *arXiv preprint arXiv:2608.00793*, 2026. 1, 2, 6, 7, 24, 30

[50] Siyuan Ma, Yutian Zhang, Boshi Zhang, Qinglian Wu, Jiaqi Zhai, Dong Wei, and Xiaojin Huang. ForeTime-VLA: Causal future-token distillation from a world action model for conveyor-belt manipulation. *arXiv preprint arXiv:2608.20735*, 2026. 2

[51] Lucas Manuelli, Wei Gao, Peter Florence, and Russ Tedrake. kPAM: Keypoint affordances for category-level robotic manipulation. In *International Symposium on Robotics Research*, 2019. 3

[52] Naresh Marturi, Marek Kopicki, Alireza Rastegarpanah, et al. Dynamic grasp and trajectory planning for moving objects. *Autonomous Robots*, 43(5):1241–1256, 2019. 2

[53] Henrique Morimitsu, Xiaobin Zhu, Roberto M. Cesar, Xiangyang Ji, and Xu-Cheng Yin. DPFlow: Adaptive optical

flow estimation with a dual-pyramid framework. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17810–17820, 2025. 17

[54] E-Ro Nguyen, Yichi Zhang, Kanchana Ranasinghe, Xiang Li, and Michael S. Ryoo. Pixel motion diffusion is what we need for robot control. *arXiv preprint arXiv:2509.22652*, 2025. 3

[55] Octo Model Team, Dibya Ghosh, Homer Walke, et al. Octo: An open-source generalist robot policy. In *Proceedings of Robotics: Science and Systems*, 2024. 2

[56] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. 1, 2, 5, 7, 24

[57] Kanchana Ranasinghe, Xiang Li, E-Ro Nguyen, Cristina Mata, Jongwoo Park, and Michael S. Ryoo. Pixel motion as universal representation for robot control. *arXiv preprint arXiv:2505.07817*, 2025. 3

[58] Kanchana Ranasinghe, Honglu Zhou, Yu Fang, Luyu Yang, Le Xue, Ran Xu, Caiming Xiong, Silvio Savarese, Michael S. Ryoo, and Juan Carlos Niebles. Future optical flow prediction improves robot control & video generation. *arXiv preprint arXiv:2601.10781*, 2026. 3

[59] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment anything in images and videos. In *International Conference on Learning Representations*, 2025. 14

[60] Yiran Shi, Dongqi Guo, Tianchen Zhao, Feng Gao, Liangzhi Shi, Chao Yu, ZhiJian Mo, Qihua Xiao, XiaoShuai Peng, Qingmin Liao, and Yu Wang. StreamingVLA: Streaming vision-language-action model with action flow matching and adaptive early observation. *arXiv preprint arXiv:2603.28565*, 2026. 2

[61] Mohit Shridhar, Lucas Manuelli, and Dieter Fox. Perceiver-actor: A multi-task transformer for robotic manipulation. In *Conference on Robot Learning*, 2022. 3

[62] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, et al. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025. 2

[63] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, et al. DINOv3. *arXiv preprint arXiv:2508.10104*, 2025. 14, 15, 30, 32

[64] Otto J. M. Smith. Closer control of loops with dead time. *Chemical Engineering Progress*, 53(5):217–219, 1957. 2

[65] Rustin Soraki, Homanga Bharadhwaj, Ali Farhadi, and Roozbeh Mottaghi. ObjectForesight: Predicting future 3D object trajectories from human videos. *arXiv preprint arXiv:2601.05237*, 2026. 3

[66] Yue Su, Sijin Chen, Haixin Shi, Mingyu Liu, Zhengshen Zhang, Ningyuan Huang, Weiheng Zhong, Zhengbang Zhu, Yuxiao Liu, and Xihui Liu. World guidance: World modeling in condition space for action generation. *arXiv preprint arXiv:2602.22010*, 2026. 3

[67] Shahram Najam Syed, Arthur Jakobsson, Haoran Hao, and Jeffrey Ichnowski. Intercepting the future: Latent-space predictive world model for dynamic VLA manipulation. *arXiv preprint arXiv:2606.02486*, 2026. 1, 2

[68] Jiaming Tang, Yufei Sun, Yilong Zhao, Shang Yang, Yujun Lin, Zhuoyang Zhang, James Hou, Yao Lu, Zhijian Liu, and Song Han. VLASH: Real-time VLAs via future-state-aware asynchronous inference. *arXiv preprint arXiv:2512.01031*, 2025. 2

[69] Zachary Teed and Jia Deng. RAFT-3D: Scene flow using rigid-motion embeddings. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021. 3

[70] Yang Tian, Sizhe Yang, Jia Zeng, Ping Wang, Dahua Lin, Hao Dong, and Jiangmiao Pang. Predictive inverse dynamics models are scalable learners for robotic manipulation. In *International Conference on Learning Representations*, 2025. 2

[71] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012. 5, 18

[72] Changhao Wang, Jaden Clark, and Shuran Song. Chronos policy: Learning robot policies on physical time. *arXiv preprint arXiv:2610.13929*, 2026. 2

[73] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. VGGT: Visual geometry grounded transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025. 3

[74] Ruicheng Wang, Sicheng Xu, Yue Dong, Yu Deng, Jianfeng Xiang, Zelong Lv, Guangzhong Sun, Xin Tong, and Jiaolong Yang. MoGe-2: Accurate monocular geometry with metric scale and sharp details. *arXiv preprint arXiv:2507.02546*, 2025. 17

[75] Yuran Wang, Siqiao Huang, Mingleyang Li, Chenhao Zhang, Jiaqi Liang, Weiyang Jin, Yue Chen, Xuemin Chi, Donghao Zhou, Qize Yu, Yu-Kai Wang, Yuhan Rui, Shenzhe Yao, Zhen Yuan, Zhenhao Shen, Kefei Zhu, Zijie Zhu, Ning Gao, Xiaowei Chi, Guanqi He, Shanghang Zhang, Hao Dong, Lin Shao, and Hang Zhao. OpenWAM: An open, modular exploration towards systematic world-action model pretraining. *arXiv preprint arXiv:2609.07398*, 2026. 1, 2, 5, 7, 24

[76] Bowen Wen, Wei Yang, Jan Kautz, and Stan Birchfield. FoundationPose: Unified 6D pose estimation and tracking of novel objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17868–17879, 2024. 3, 14

[77] Chuan Wen, Xingyu Lin, John So, Kai Chen, Qi Dou, Yang Gao, and Pieter Abbeel. Any-point trajectory modeling for policy learning. In *Proceedings of Robotics: Science and Systems*, 2024. 3

[78] Gaoyuan Wu, Ziyu Shan, Haoyang Du, Yuyao Jiang, and Ziwei Wang. AffordTrajDP: Dynamic affordance-guided

visuomotor policy learning for robotic manipulation. *arXiv preprint arXiv:2608.01603*, 2026. 3

[79] Hongtao Wu, Ya Jing, Chilam Cheang, Guangzeng Chen, Jiafeng Xu, Xinghang Li, Minghuan Liu, Hang Li, and Tao Kong. Unleashing large-scale video generative pre-training for visual robot manipulation. In *International Conference on Learning Representations*, 2024. 2

[80] Yuxi Xiao, Qianqian Wang, Shangzhan Zhang, Nan Xue, Sida Peng, Yujun Shen, and Xiaowei Zhou. SpatialTracker: Tracking any 2D pixels in 3D space. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024. 3

[81] Haozhe Xie, Beichen Wen, Jiarui Zheng, Zhaoxi Chen, Fangzhou Hong, Haiwen Diao, and Ziwei Liu. DynamicVLA: A vision-language-action model for dynamic object manipulation. *arXiv preprint arXiv:2601.22153*, 2026. 2, 5, 7, 24

[82] Mengda Xu, Zhenjia Xu, Yinghao Xu, Cheng Chi, Gordon Wetzstein, Manuela Veloso, and Shuran Song. Flow as the cross-domain manipulation interface. In *Conference on Robot Learning*, 2024. 3

[83] Zhenjia Xu, Zhanpeng He, Jiajun Wu, and Shuran Song. Learning 3D dynamic scene representations for robot manipulation. In *Conference on Robot Learning*, 2020. 3

[84] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026. 1, 2

[85] Hanyang Yu, Haitao Lin, Jingbo Zhang, Wenyao Zhang, Chenghao Gu, Heng Li, and Ping Tan. MaskWAM: Unifying mask prompting and prediction for world-action models. *arXiv preprint arXiv:2606.13515*, 2026. 3

[86] Heng Yu, David D. Yuan, Juze Zhang, Changan Chen, Yao Feng, Michelle Baldonado, Steve Cousins, Li Fei-Fei, Jiajun Wu, and Ehsan Adeli. OpenWAM: An open framework for composable world-action models. *arXiv preprint arXiv:2610.07922*, 2026. 2

[87] Chengbo Yuan, Chuan Wen, Tong Zhang, and Yang Gao. General flow as foundation affordance for scalable robot learning. In *Conference on Robot Learning*, pages 1541–1566, 2024. 3

[88] Shanglin Yuan, Weiheng Zhao, Xin Shi, Haoyi Jiang, Xianda Guo, Liu Liu, Wenyu Liu, Wei Sui, and Xinggang Wang. DreamWAM: Beyond RGB future prediction for world action models. *arXiv preprint arXiv:2608.04996*, 2026. 3

[89] Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-WAM: Do world action models need test-time future imagination? *arXiv preprint arXiv:2603.16666*, 2026. 3

[90] Peng Yun, Shouwang Huang, Hao Li, Jinxi Li, Jianan Wang, and Bo Yang. PhysMani: Physics-principled 3D world model for dynamic object manipulation. *arXiv preprint arXiv:2607.01938*, 2026. 2

[91] Yanjie Ze, Gu Zhang, Kangning Zhang, Chenyuan Hu, Muhan Wang, and Huazhe Xu. 3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations. In *Proceedings of Robotics: Science and Systems*, 2024. 3

[92] Chuhan Zhang, Guillaume Le Moing, Skanda Koppula, Ignacio Rocco, Liliane Momeni, Junyu Xie, Shuyang Sun, Rahul Sukthankar, Joëlle K. Barral, Raia Hadsell, Zoubin Ghahramani, Andrew Zisserman, Junlin Zhang, and Mehdi S. M. Sajjadi. Efficiently reconstructing dynamic scenes one D4RT at a time. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026. 3

[93] Jiahui Zhang, Yurui Chen, Yueming Xu, Ze Huang, Yanpeng Zhou, Yu-Jie Yuan, Xinyue Cai, Guowei Huang, Xingyue Quan, Hang Xu, and Li Zhang. 4D-VLA: Spatiotemporal vision-language-action pretraining with cross-scene calibration. In *Advances in Neural Information Processing Systems*, 2025. 3

[94] Wenyao Zhang, Hongsi Liu, Zekun Qi, Yunnan Wang, Xin-Qiang Yu, Jiazhao Zhang, Runpei Dong, Jiawei He, He Wang, Zhizheng Zhang, Li Yi, Wenjun Zeng, and Xin Jin. DreamVLA: A vision-language-action model dreamed with comprehensive world knowledge. In *Advances in Neural Information Processing Systems*, pages 24195–24228, 2025. 1, 3, 5, 7, 24

[95] Yifan Zhang, Ruiping Wang, and Xilin Chen. Dynamic behavior cloning with temporal feature prediction: Enhancing robotic arm manipulation in moving object tasks. *IEEE Robotics and Automation Letters*, 10(6):5209–5216, 2025. 2

[96] Qingqing Zhao, Yao Lu, Moo Jin Kim, Zipeng Fu, Zhuoyang Zhang, Yecheng Wu, Zhaoshuo Li, Qianli Ma, Song Han, Chelsea Finn, Ankur Handa, Tsung-Yi Lin, Gordon Wetzstein, Ming-Yu Liu, and Donglai Xiang. CoT-VLA: Visual chain-of-thought reasoning for vision-language-action models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025. 2

[97] Ruiteng Zhao, Zhengshen Zhang, Yue Su, Wenshuo Wang, Jiahui Li, Zhiyuan Yang, Francis E. H. Tay, Marcelo H. Ang, and Haiyue Zhu. SG-WAM: Self-guided world modeling in geometry-aware policy space. *arXiv preprint arXiv:2608.01397*, 2026. 3

[98] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems*, 2023. 2, 5, 7, 24

[99] Haoyu Zhen, Xiaowen Qiu, Peihao Chen, Jincheng Yang, Xin Yan, Yilun Du, Yining Hong, and Chuang Gan. 3D-VLA: A 3D vision-language-action generative world model. In *International Conference on Machine Learning*, 2024. 3

[100] Ruijie Zheng, Yongyuan Liang, Shuaiyi Huang, Jianfeng Gao, Hal Daumé III, Andrey Kolobov, Furong Huang, and Jianwei Yang. TraceVLA: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies. In *International Conference on Learning Representations*, 2025. 3

[101] Hongyan Zhi, Peihao Chen, Siyuan Zhou, Yubo Dong, Quanxi Wu, Lei Han, and Mingkui Tan. 3DFlowAction: Learning cross-embodiment manipulation from 3D flow world model. *arXiv preprint arXiv:2506.06199*, 2025. 3

[102] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features

enable zero-shot planning. In *International Conference on Machine Learning*, 2025. 2

[103] Chuning Zhu, Raymond Yu, Siyuan Feng, Benjamin Burchfiel, Paarth Shah, and Abhishek Gupta. Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets. In *Proceedings of Robotics: Science and Systems*, 2025. 2

[104] Yuke Zhu, Josiah Wong, Ajay Mandlekar, Roberto Martín-Martín, Abhishek Joshi, Kevin Lin, Abhiram Maddukuri, Soroush Nasiriany, and Yifeng Zhu. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*, 2020. 5, 18

[105] Yifeng Zhu, Zhenyu Jiang, Peter Stone, and Yuke Zhu. Learning generalizable manipulation policies with object-centric 3D representations. In *Conference on Robot Learning*, 2023. 3

[106] Brianna Zitkovich, Tianhe Yu, Sichun Xu, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, 2023. 1, 2

Supplementary Material

This supplement gives implementation details of SE(3)-WAM (Appendix A), pretraining on real robot data (Appendix B), the benchmark (Appendix C), the baselines, DOMINO comparison, real robot, and timing (Appendix D), additional results (Appendix E), the complete ablation study behind Sec. 4.4 (Appendix F), and the error analysis that motivates the design (Appendix G). Colored dashes and bracketed entries denote values pending the final runs.

A. Implementation

A.1. Scene and Summary Tokens

Current image. A frozen MAE ViT-B/16 [26] encodes the 224×224 agent-view RGB into 196 patch tokens. We selected it over frozen DINOv3 encoders [63] and a CNN trained with the policy on development scenes (Appendix F.1). Each patch receives a view embedding, a linear embedding of its mean valid metric depth, and a linear embedding of the back-projected base-frame position of its center pixel, which we compute with the camera intrinsics and extrinsics. These positions also enter 3D rotary position embeddings for the patch queries and keys. The same center points are the source points of the sparse 3D flow (Appendix A.3). Target, robot, summary, and action tokens receive no rotary embedding. Past images never enter the model, and we drop their slots before the backbone, so the scene holds 212 tokens.

Target-motion history. At each query, the target history covers the last 8 control steps, the offsets $-350, -300, \ldots, 0$ ms. Each of the 8 target-state tokens is a 25-dimensional descriptor $(\mathbf{p}, \log R, \Delta\mathbf{p}, \log \Delta R, \Delta t, \mathbf{v}, \boldsymbol{\omega}, \mathbf{a}, \boldsymbol{\alpha})$ with dimensions $(3, 3, 3, 3, 1, 3, 3, 3, 3)$. Slots 0–6 describe the seven intervals between consecutive offsets, and slot 7 holds the current absolute pose in the base frame. Velocities divide interval displacements by the elapsed time, and accelerations divide adjacent velocity differences by the interval-center spacing. We standardize each dimension with training statistics, zero the invalid dimensions, and append three validity flags (pose, interval, and acceleration) before a $28 \to 1024 \to 1024$ GELU MLP with an additive relative-time embedding. Invalid features are zeroed; a slot with no valid feature is masked out, without a learned lost-target token. In simulation, the poses come from the simulator. On the real robot, they come from the causal tracker of Appendix A.2, whose track we resample at the same offsets, interpolating linearly in translation and along the geodesic in rotation, without extrapolation and without interpolating across gaps longer than two camera periods.

Table S1. **Selected marker-free pipeline.** Median/p95 translation (mm) and rotation (°) error against a stated marker-free reference, its uncertainty, and the fraction of lost observations. Real-ball rotation is excluded. Measurements and the reference procedure remain pending; estimated labels are not ground truth.

Target	mm	°	Ref. uncertainty	Lost
Real rigid object	—	—	—	—
Real ball	—	–	—	—

Robot state. 8 end-effector states (position, rotation vector, and measured gripper opening fraction) at the same offsets form the robot tokens. The policy has no language token; view, time, depth, and position embeddings modify existing tokens rather than adding slots.

Summary tokens. 16 learned embeddings form the summary tokens of the world stream. They attend to the scene tokens and to each other at every layer, and they are the only world tokens that the action tokens attend to.

A.2. Pose Tracker

The real-robot protocol is fully marker-free, including online tracking, offline training labels, and error measurement. Each object has one fixed reference with a shared origin, axes, scale, and instance identity across demonstrations. The candidate pipeline uses causal masks from Grounding DINO and SAM 2.1 [44, 59] and RGB-D registration to this reference, with FoundationPose [76] as a candidate. [Validate and specify the tracker, reference acquisition, image resolution, and loss/recovery thresholds on the physical setup.]

Policy inputs must be obtained by causal replay of the deployment tracker. Each estimate carries its capture time, availability time, and component validity; later frames must not revise an estimate already issued to the policy. Offline supervision may use multiframe or multiview registration and manual review in the same object frame. These estimates are not independent ground truth. Tracker error measurements require a marker-free reference with stated uncertainty; reprojection consistency alone does not provide that reference.

For real-ball trials, the protocol excludes target orientation, relative rotation, angular velocity, and angular acceleration from policy inputs, and target rotation from forecast supervision and reported errors. Ball-center translation, translational velocity and acceleration, and their valid observations are retained. The mask does not discard the full target-state slot or change robot orientation states, rotation commands, or the full SE(3) simulation experiments. No ball-orientation observability study is required. Table S1 records the selected pipeline's measurements when available. Actual error calibration and tracked closed-loop evaluation remain separate from the synthetic-noise study of Appendix F.2.

A.3. Forecast Heads

All forecast heads are light readouts of the final hidden states of the world stream. They are trained jointly with the action and skipped at deployment, and no token reads their outputs, so they shape the world stream only through the losses.

Target. A linear map from the mean final state of the 16 summary tokens reads out the trajectory $\{(\mathbf{d}_h, \boldsymbol{\phi}_h)\}_{h=1}^{8}$. With valid horizons $\mathcal{V}$,

$$\mathcal{L}_P = \frac{1}{|\mathcal{V}|} \sum_{h \in \mathcal{V}} \Big[\|\hat{\mathbf{d}}_h - \mathbf{d}_h\|^2 + \lambda_R \, d_{\mathrm{SO}(3)}\big(\hat{R}_h, R_h\big)^2 \Big], \quad \text{(S1)}$$

with $\lambda_R = 0.1$ and $\hat{R}_h = \mathrm{Exp}(\hat{\boldsymbol{\phi}}_h^{\wedge}) R(\tau_t)$. We use the left-relative rotation vector and the cumulative translation instead of a twist exponential, which would couple translation to rotation through its Jacobian. Squared-error training yields a point estimate and does not represent multiple future modes explicitly.

Sparse 3D flow. The flow is the cumulative 3D displacement of the $N = 196$ patch center points over the 8 horizons, $F \in \mathbb{R}^{N \times 8 \times 3}$, in the base frame, together with a visibility $o \in \{0,1\}^{N \times 8}$. A linear readout on each patch token gives its point's displacement and visibility logit at every horizon. We divide the displacements by a fixed training-label scale $s_F = 0.7154605\,\mathrm{m}$ and use

$$\mathcal{L}_F = \frac{1}{|\mathcal{V}_F|} \sum_{(i,h) \in \mathcal{V}_F} \Big[\frac{\|\hat{f}_{ih} - F_{ih}/s_F\|^2}{3} + \mathrm{BCE}(\hat{o}_{ih}, o_{ih}) \Big], \quad \text{(S2)}$$

where $\hat{f}$ is the normalized displacement and $\mathcal{V}_F$ holds valid labels. Visibility BCE has weight 1 in simulation; invisible but valid points still supervise displacement. The points cover the target, the robot, and the context alike; the target's precision comes from its pose head.

Appearance targets. Linear readouts of the final patch tokens predict three targets at every horizon: metric depth and moving-surface RGB through 16×16 pixel readouts per patch, and 1024-dimensional features of a frozen DINOv3 ViT-L/16 teacher [63] on the 14×14 grid. Depth uses squared error on valid pixels; DINO features use squared error after channel-wise layer normalization; RGB uses squared error on surfaces that move by more than 1 mm, including the robot. These losses average channels and then valid positions. A $1024 \to 256 \to 3072$ MLP with GELU and a final sigmoid reads the mean of the final summary tokens to reconstruct the current target's 32×32 RGB region, using absolute error averaged over valid RGB elements. This ties the summary features to the target without adding an input. The four losses form $\mathcal{L}_{\mathrm{aux}}$; $\lambda_{\mathrm{aux}} = 0.1$ gives each a weight of 0.1 in the total objective. All four heads are used only in training and skipped at deployment; their outputs never enter the action stream.

A.4. Action Stream

Each chunk has 8 seven-dimensional commands, each with a base-frame translation increment, a left-relative rotation vector, and a gripper command, at a control period of 50 ms. Translation and nonzero rotation increments reference the measured end-effector pose at application; a zero rotation command can retain the controller's preceding orientation goal. Normalization uses the mean and the population standard deviation of valid, non-repeated issued commands in the training split. Flow matching uses a unit Gaussian in these coordinates with one u per chunk, and the action tokens receive u through adaptive layer normalization. Each prediction position draws 8 independent noise/time pairs for its chunk and averages their losses, which reduces the variance of the action gradient without enlarging the batch. At deployment, we integrate 5 Euler steps, reverse the normalization, decode the rotation vectors, and clip each axis to the exported motion limits: 0.05 m and 0.5 rad in the candidate configuration; 0.10 m translation requires an explicit deployment override. The gripper outputs 0 when $|g| \le b$ and otherwise $\mathrm{sign}(g)$, where $b = 0$ by default. The separately validated E5 deployment uses $b = 0.5$ for catching and $b = 0.1$ otherwise; it is not the final candidate's configuration.

A.5. Backbone and Coupling

Backbone. Each of the 24 layers applies one attention to all tokens with shared Q/K/V and output projections, followed by separate SiLU feed-forward networks (ratio 4) for the world tokens and the action tokens. World tokens use layer normalization and action tokens adaptive layer normalization, for 504.0M trainable backbone parameters. The alternative of Appendix F.1 gives the two streams separate attention projections, normalization, and feed-forward networks at a matched parameter count.

Attention mask. The mask acts on three token groups, the scene ($\mathcal{S}$), the summary tokens ($\mathcal{M}$), and the actions ($\mathcal{A}$). Scene tokens attend only to $\mathcal{S}$; summary tokens attend to $\mathcal{S}$ and $\mathcal{M}$, and action tokens attend to $\mathcal{M}$ and $\mathcal{A}$. A blocked edge has the logit $-\infty$ at every layer and head, both in training and at deployment. Because the world tokens attend neither to the actions nor to the flow time, their states are computed once per query and reused across the Euler steps, and no future label reaches the action except through the gradients of the forecast losses on the shared parameters.

Screening couplings. The 4-layer variants of Appendix F.4 use the same modules, initialization, and losses and change only the mask or add a small module. R0 disables every forecast loss. R1 adds one object query and one query per patch to the world stream, which attend to $\mathcal{S}$, $\mathcal{M}$, and each other and carry the forecast readouts instead of the pooled states, but opens no edge between them and the actions. R2 lets the action tokens attend to these queries at every layer. R3 lets these queries attend to the actions, and R4 opens both edges; in R3 and R4, the world states are recomputed at every Euler step. R5 decodes the forecast once per query and encodes the target trajectory and the flow with visibility probabilities, with rotations in a continuous 6D form, through a geometry encoder (0.37M parameters) into geometry tokens that the action attends to in addition to $\mathcal{M}$. R5 trains on the predicted geometry and backpropagates the action loss through it without teacher forcing. We detach no gradient between the streams in any variant.

A.6. Training Recipe

Table S2 lists the recipe for training on Dynamic-14. Every control step of a demonstration is a query window, which gives 1,324,374 training windows from the 11,200 demonstrations, and 350 held-out episodes serve as the validation split. Each loss is normalized by its count of valid labels over the whole accumulated global batch. Each window contains 8 independent query positions with their own causal histories; there is no attention across these positions. The image encoder processes the current frame, and the DINOv3 teacher's features of the future frames are cached once per episode. The final simulation training is underway; its completed-run duration and hardware remain to be reported: [X] hours on [N] RTX PRO 6000 GPUs. The DROID base uses the recipe of Appendix B.4.

A.7. Deployment

Each query encodes the newest observation, runs the world stream once without its heads, and integrates the action stream on the cached world states. The chunk starts executing at the first control-period boundary at or after the query ends, and the next query starts at that same boundary with a new observation, so the policy replans as often as its latency allows. Under the real-time protocol of Appendix C.3, physics advances while a query runs and the executor continues with the remainder of the previous chunk. Appendix C.3 describes the executor shared by all methods, and Appendix D.4 the timing measurements.

Table S2. Training hyperparameters on Dynamic-14.

Item	Value
Backbone	24 layers, width 1024, 16 heads, FFN 4096, 504.0M parameters
Tokens	212 scene, 16 summary, 8 action
Encoder	frozen MAE ViT-B/16, 224×224 input
Optimizer	AdamW, $\beta = (0.9, 0.999)$, $\epsilon = 10^{-8}$, weight decay 10^{-4}
Learning rate	2×10^{-4}, 5,174 warm-up updates from 0, cosine decay to 0
Batch, updates	global batch 512 windows, 20 epochs (51,740 updates)
Loss weights	$\lambda_P = \lambda_F = 1$, $\lambda_R = 0.1$, $\lambda_{\text{aux}} = 0.1$ per target
Precision	trainable weights FP32, frozen encoder BF16, gradient clipping at 0.1
Action noise	8 noise/time pairs per query position
Sampling	5 Euler steps for the action
Inference	FP32, world states cached within a query
Hardware	[N] RTX PRO 6000 GPUs, about [X] hours per run

B. Pretraining on Real Robot Data

B.1. Data, Partitions, and Time

The pretraining corpus is DROID [36], recorded at 15 Hz with two exterior cameras and one wrist camera, together with the 3D point tracks and optimized camera calibrations that PointWorld released for it [30]. We associate each released clip with its raw DROID episode by requiring exact agreement of the joint sequence, end-effector poses, gripper states, commands, and camera identity, and we exclude clips whose capture clock is not monotonic or whose association is ambiguous. Of 42,935 associated episodes, 42,087 yield usable windows. The model sees the first exterior camera, its depth, and PointWorld's calibration of it. The training unit is a causal observation window with its aligned future and action labels, 598,889 anchor windows in total: 538,740 from 37,874 training episodes and 60,149 from 4,213 validation episodes. The partitions share no episode. Table S3 reports the yield and the validity of the labels.

Time. All times are capture timestamps, and we do not relabel frames to a nominal rate. The robot history uses the 8 most recent states at or before the query time, causally, as at deployment. Flow labels keep PointWorld's own record times rather than being interpolated to the 50 ms horizons of the simulation model. The future DINOv3 targets use the frames at the same record times. Each anchor expands into 8 consecutive native query positions with independent causal histories. Only the anchor position retains XYZ and DINO supervision; the remaining positions provide action supervision without interpolated or duplicated world labels.

Table S3. **DROID pretraining data.** Yield of each stage and validity of the labels. Windows are causal observation windows with aligned labels, one per control step of the main camera. Flow labels count point–horizon slots.

	Train	Val.	All
Episodes associated	–	–	42,935
Episodes with windows	37,874	4,213	42,087
Windows	538,740	60,149	598,889
Valid actions (%)	—	—	96.88
Valid flow labels (%)	—	—	47.05
Future frames for DINO	—	—	3,425,218
Windows with moving objects (%)	—	—	—

Actions. We express robot states and Cartesian commands in the robot base frame after the calibrated flange-to-TCP transform, with the parameterization of Appendix A.4. We admit a label if its values are finite and its reference state was available causally. The robot's execution limits are not an admission rule, because a per-axis limit would discard fast but valid motion and bias the base toward slow demonstrations. Of 4,791,112 action slots, we mask the 149,318 at which DROID's collector issued no command and keep the remaining 96.88%, with no filter on magnitude or speed. Action and flow validity are independent, so an invalid action does not remove the flow labels of its window.

B.2. What the Base Learns

DROID provides no object poses, and its instructions name the manipulated object without marking it. The base therefore has no target-state tokens and no pose head. Its inputs are the current RGB-D frame and the robot history (204 scene tokens), with unavailable depth explicitly masked, and its objectives are the action loss (weight 1), the sparse 3D flow of the selected patch tracks (weight 1), and the future DINOv3 features (weight 0.1). Future depth, moving-region RGB, and the target reconstruction are off. Because PointWorld's tracks record only visible positions, the flow labels contain no negative visibility examples, and we set the visibility loss to zero rather than teach the model that every point stays visible. Most objects in DROID move only when the robot moves them, so the base mainly learns metric geometry, the 3D motion that the robot's own actions cause, and its coupling with the actions; the exogenous motion of our tasks and the target's pose enter during fine-tuning.

B.3. Flow Labels

PointWorld's tracks are already expressed in the base frame. Each patch of the 14×14 grid follows one fixed track over the window. Among the released tracks whose initial projection lies inside the patch, are initially visible, and have a valid positive depth, we prefer the one with the largest image motion and replace it by another candidate when its 3D displacement disagrees with its image motion. Otherwise we take the track nearest to the patch center. This rule covers moving objects that a fixed center point would miss: in a check of 30 segments from 10 episodes, 18 of which show visible object motion, it placed the start and two future points of a dynamic track inside the object in 13 of these 18 segments, against 5 for the center rule. This is coverage of image-consistent motion, not a measured 3D accuracy, and a patch on a small object can still drift to the background. The label is the track's displacement from its initial position at each record time. Patches without a valid track are invalid, and we do not fill them from neighbors. Over all windows, 47.05% of the point–horizon slots carry a valid label.

Label quality. We compared the PointWorld tracks with flow lifted from 2D correspondences [53] with monocular metric depth [74] on the same scenes. On surfaces that are static in the image, the selected PointWorld tracks move by [0.9] and [5.8] mm (median and 95th percentile) over the horizons, against [18] and [72] mm for flow lifted with monocular depth, whose scale drifts over time, so we use the PointWorld tracks and do not mix in the lifted flow. PointWorld's masks are not reliable enough to separate objects from the background, so we do not use them.

B.4. Recipe and Scale

Simulation model. SE(3)-WAM trains on Dynamic-14 from the generic visual initialization with the recipe of Table S2. We chose its depth among 12, 24, and 48 layers at width 1024 and its recipe with short runs at equal numbers of training samples, selected on development scenes (Fig. S2a).

DROID base. The base uses the same architecture family, with 48 layers at width 1024 (1.02B trainable parameters), and only real data. Its configured budget is 20 epochs (21,060 updates, 2,106 warm-up updates) at a global batch size of 512 on 64 GPUs, with the optimizer and precision of Table S2. Pretraining is ongoing; the final checkpoint is not yet available (Fig. S2b).

B.5. Task-Specific Fine-Tuning

Each real-robot policy is initialized from the base and fine-tuned on the demonstrations of its task only. The target-state tokens, the pose head, and the appearance heads that the base lacks are added with fresh initialization, and the policy receives poses from the tracker of Appendix A.2. Simulation weights do not initialize any real-robot policy, and the base does not see simulation data. Each policy creates its own optimizer, schedule, training-only normalization, and readouts.

Figure placeholder: Fig. S1 — flow labels from PointWorld tracks (schematic)

Left: one DROID frame of the main exterior camera with PointWorld's optimized camera pose next to the raw one, and the 14×14 patch grid. **Middle:** in each patch, the candidate PointWorld tracks (thin gray) and the selected track (orange): the one with the largest image motion among the tracks that stay consistent in 3D, falling back to the track nearest the patch center; patches without a valid track greyed out. **Right:** the selected tracks' 3D displacements in the base frame at PointWorld's record times, with the robot's gripper path in purple. Use one real DROID frame sequence, vector overlays only.

Figure S1. Flow labels for the DROID base. Each image patch follows one PointWorld track, selected to cover moving objects where they exist, and its 3D displacement in the base frame is the label. Invalid labels mask their losses.

Figure placeholder: Fig. S2 — recipe and scale

(a) Simulation model. y: development SR; x: 12, 24, and 48 layers at equal sample exposure; the selected configuration starred. **(b) DROID base.** x: training samples (log); y: validation flow endpoint error at the last horizon (left axis, mm) and action loss (right axis) for the 48-layer run; marker at the selected checkpoint.

Figure S2. Recipe and scale of the simulation model (a) and the DROID base (b).

C. Dynamic-14

C.1. Tasks, Roles, and Terms

We built Dynamic-14 on MuJoCo and robosuite [71, 104]. It contains fourteen tasks in five families (Fig. S3). Table S4 lists their success conditions and Table S6 their scenario ranges. We will release the tasks, the accepted scenarios, the success checks, and the demonstrations.

Roles and critical leads. Table S5 assigns the bodies of each task to the two roles of Sec. 3.1. The target is the body whose pose error enters a contact or containment tolerance ε, and the context consists of bodies that only need to be avoided with clearance m. A moving receptacle is therefore a target, since its pose decides containment. The wire of the passage tasks plays both roles. Its pose at the entry is a target, because the wand must be aligned within 30 mm there, and its geometry along the loop is context, because any contact fails. The table also gives each task's speeds and its critical lead $\Delta^\star = \varepsilon/\|\mathbf{v}\|$ (Appendix G.5), and its last column lists the error terms that each task exercises (Eq. 3 and Appendices G.2–G.3). Conveyor and translating targets set the speed and the lead. Rotating targets, SE(3) placement, and the wire make orientation enter the contact. Ramps, low friction, and strikers produce motion that extrapolation misses, the occluder, wire, tray, and striker set the clearance, and the occluder also removes history.

We check success at every 2 ms physics step, after the ordered requirements of Table S4, and an interrupted continuous condition resets its counter. Terminal events fall into five categories that we report separately. An *invalid* event is a corrupted simulator state. *Boundary* events are a workspace or downstream exit, a drop, or an expired goal window. *Safety* events are unsafe contact, an excessive rim impulse, wire contact, a release impulse, early robot contact, unsupported tray contact, or a striker trap. *Task violations* are a release outside the goal, a path deviation or backtracking, a premature release, a deflection without capture, tipping, a missing or extra impact, or an insufficient change of outgoing motion. The last category is the *timeout*. Each family evaluates its events in a fixed order per physics step and keeps the first terminal event. The task-balanced score weights every task equally, including tasks on which some methods never succeed.

C.2. Scenarios and Demonstrations

We sample continuous ranges uniformly and discrete choices with equal probability, and then apply physical-feasibility checks. Evaluation replays the stored accepted scenarios. Harmonic translation follows $\mathbf{p}(t) = \mathbf{c} + (0, v_y, 0)t + (A_x \sin(2\pi f_x t + \phi), 0, 0)$ with $\phi \sim \mathcal{U}[-\pi, \pi]$. Capture targets start at rest and move under gravity and friction on ramps of $[5, 15]^\circ$. We accept an occluded scene only if the target is visible for at least eight initial frames, fully occluded for 0.5–3.0 s, and visible again for at least three frames.

The training pool contains 11,200 demonstrations, 800 per task. They were generated by experts with privileged simulator state, whereas the learned policies receive only the inputs of Sec. 3.2. We record the demonstrations at 20 Hz with camera images, depth, robot states, issued commands, and simulator ground truth for target poses and scene geometry. From these we compute the future labels, their validity masks, and the command labels. The policies receive the target pose from the simulator, which the pose-noise sweep perturbs (Appendix C.3).

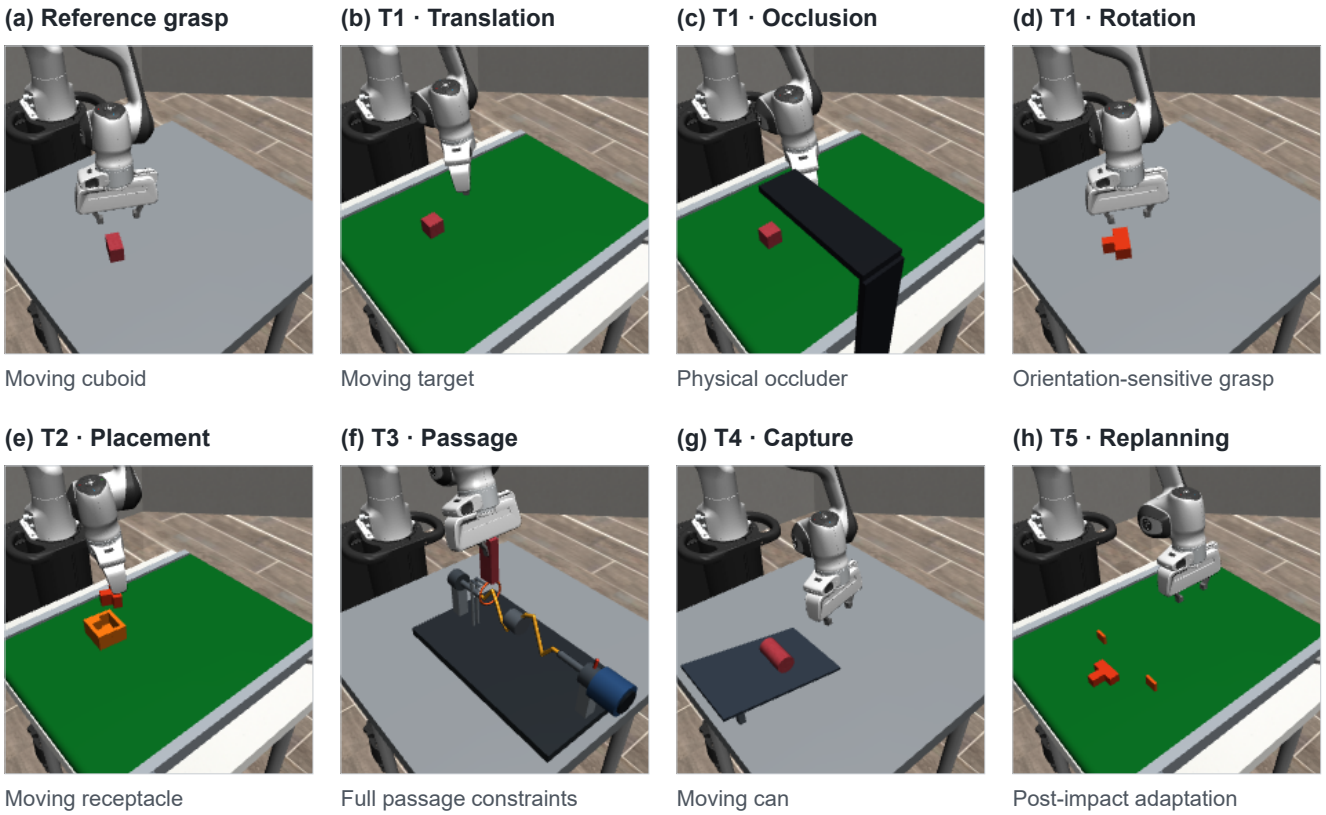


Figure S3. Dynamic-14 observations. Eight examples cover reference grasping, grasping a translating, occluded, or rotating target, placement into a moving receptacle, passage along a rotating wire, capture of a rolling object, and grasping after an impact. Each panel is the final observed, uncropped 224×224 agent-view RGB of one development scene. Table S4 lists all fourteen tasks.

C.3. Protocol and Sweeps

Executor. Every method runs under the same asynchronous executor on a 50 ms control clock, and queries start on its period boundaries. The next query starts at the first boundary at or after the end of the previous inference, which is also when the chunk of that inference starts executing. While a query runs, the executor continues with the remainder of the previous chunk, one command per control period, and the new chunk replaces this remainder. A method whose inference fits in one control period therefore replans before every command, and a method with latency ℓ replans every $\max(1, \lceil \ell / 50\,\mathrm{ms} \rceil)$ control periods. Before the first chunk of an episode arrives, and whenever a chunk runs out before the next one, the controller holds the current arm target and gripper command while physics advances; no command is repeated. A baseline whose paper replans less often keeps its own chunk length under the *original executors* protocol.

Real-time protocol. Unless noted otherwise, physics advances during inference by a latency drawn per query from the method's measured latency distribution on one RTX 5090 (Appendix D.4). The post-impact grasp additionally has a 50 ms command delay built into the task. Table S14 reports three alternatives. With *paused* physics, the shared executor is kept and the simulation waits for inference; any added artificial delay still advances physics. With *equal latency*, every method is charged the same latency, the p95 of SE(3)-WAM, regardless of its own speed. With *original executors*, each baseline keeps the execution scheme of its paper.

Latency. An added delay of 0, 50, 100, 150, or 200 ms is inserted after inference on top of the protocol's latency, and physics advances during it.

Pose noise. With frozen weights, we add three synthetic conditions: nominal temporally correlated pose noise, twice its amplitude, and nominal noise with contiguous missing observations. Perturbations affect current and historical estimates before caching and kinematics, with the clean real-time result reused as reference (Appendix F.2). These conditions are not calibrated to a physical tracker; actual calibration and tracked closed-loop evaluation remain pending.

Table S4. Success conditions of the fourteen Dynamic-14 tasks, with the thresholds of our task implementations. A task must pass its preceding stages and avoid terminal failures. All tasks run at 20 Hz with a 2 ms physics step; durations below refer to physical time. COM denotes center of mass.

Task	Ordered success conditions	Limit
grasp; t1_belt, t1_xyz, t1_so3, t1_occ	Approach within 0.10 m; bilateral fingerpad contact for 0.05 s; collision-geometry bottom at least 0.05 m above support; retain contact and clearance for 0.50 s. T1 adds conveyor drive, translation, rotation, or a physical occluder. SO3 has no additional success-angle threshold; unsafe occluder contact terminates OCC.	160 steps 8 s
t2_trans	Approach within 0.10 m. Entire object collision geometry inside the receptacle: planar clearance and floor gap both ≥ -0.002 m, top no higher than the opening. Release requires an opening command < -0.5 and no fingerpad contact. Relative linear/angular speeds ≤ 0.08 m/s and 0.8 rad/s for 0.10 s, followed by a 0.50 s stable dwell.	200 10 s
t2_se3	Approach within 0.05 m and 0.12 rad. Full T-shaped containment: planar clearance ≥ 0.002 m, floor gap in $[-0.002, 0.002]$ m, tilt ≤ 0.05 rad, top below opening. After release, relative speeds ≤ 0.02 m/s and 0.04 rad/s for 0.10 s, then a 0.50 s dwell. The 0.12 rad gate applies to approach.	200 10 s
t3_phase	The episode starts with the wand already grasped. Phase alignment: distance ≤ 0.090 m and loop-direction error ≤ 0.20 rad. Enter within 0.030 m at progress ≤ 0.15; reach historical maximum progress ≥ 0.60, then ≥ 0.85 and endpoint distance ≤ 0.055 m.	420 21 s
t3_full	Acquire the wand with bilateral contact for 0.05 s, then complete the phase stages above. In a later sample, retain current progress ≥ 0.85 and move at least 0.045 m from both path and endpoint. Finally open (< -0.5) with no fingerpad contact. Releasing before exit fails.	640 32 s
t4_sphere	Approach within 0.10 m; first qualifying contact after ramp exit; any tray contact before success fails. Bilateral hold for nominal 25 ms (12 physics samples, 24 ms). Raise COM by 0.05 m from its height when the bilateral-hold criterion is first met, and retain contact and height for 0.50 s.	120 6 s
t4_sphere_tabletop; t4_slider	Approach within 0.10 m; contact after ramp exit, while supported, in the capture region, and moving at planar speed ≥ 0.08 m/s. Bilateral hold for 0.05 s, COM lift ≥ 0.05 m from the first attainment of that hold criterion, then contact and height for 0.50 s.	120 6 s
t4_can	Same supported-capture sequence. Before first fingerpad contact, a can-axis absolute dot product with vertical > 0.45 fails as unstable tipping. The configuration does not require a transverse closing axis.	120 6 s
t5_replan	Exactly one selected-side impact group completes before robot contact. At 30 ms after the physical impact-end timestamp, branch-signed lateral speed ≥ 0.035 m/s and yaw rate ≥ 0.35 rad/s. Approach within 0.10 m; bilateral hold with grasp-axis error $\leq 18^\circ$ for 0.05 s; COM lift ≥ 0.05 m from the first attainment of that hold criterion and stable hold for 0.50 s. Impact-group lag is 12 ms and maximum contact is 0.10 s.	120 6 s

C.4. Splits and Statistics

We select final configurations on development scenes. The main Dynamic-14 comparisons and final-model ablations use a separate held-out set of 1,400 scenes, generated with the same ranges and with new source seeds that appear in no training, validation, or development episode. The first 100 feasible scenes in increasing seed order form each task's set. The screening results in Appendix F retain their original splits and execution protocols.

The full model, CV-input variant, and core ablations in Table 3 are assigned training seeds 0, 1, and 2. Screening studies retain their recorded seed counts. Baselines use a single training seed (Table S7). On development scenes, three training seeds of the same configuration differed by up to [7.4] points in SR. Most of this spread came from the wire passage, whose success ranged from [0] to [71]% across seeds, and we do not interpret single-seed differences of a few points. We report each training seed and the cross-seed mean and standard deviation. We also average the seeds per scene and bootstrap held-out scenes within each task, jointly across methods, for paired 95% intervals. These intervals quantify scene uncertainty conditional on the trained weights; training-seed variation is reported separately. We treat differences whose interval lies within ± 2 points as ties.

C.5. Metrics

The task-balanced success rate is

$$\mathrm{SR}(m) = \frac{1}{14} \sum_{k=1}^{14} \frac{1}{n_k} \sum_{i=1}^{n_k} y_{mki}, \tag{S3}$$

for binary outcomes y_{mki} of method m on scene i of task k. The *feasible contact window* of a scene is the set of control-

Table S5. **Roles, tolerances, and critical leads of the Dynamic-14 tasks.** The target is the body whose pose error enters a contact or containment tolerance ε. The context comprises bodies that must only be avoided, with clearance m. Speeds follow the sampling ranges of Table S6 (capture tasks: at the ramp exit), and the critical lead $\Delta^\star = \varepsilon/\|\mathbf{v}\|$ is given for the median speed. Terms: error terms that a task exercises (Eq. 3 and Appendices G.2–G.3). v: moving target. $r\theta$: orientation enters the contact. e_{dyn}: non-constant motion (harmonic, gravity, friction, impact). m: a context body must be avoided. Occ.: history gaps. Delay: command delay built into the task.

Task	Code	Target	Context	ε (mm)	$\|\mathbf{v}\|$ (m/s)	$\Delta^\star$ (ms)	m (mm)	Terms
Harmonic grasp	grasp	block	table	[X]	0.03–0.20	[X]	–	v, $r\theta$, e_{dyn}
Conveyor grasp	t1_belt	part	belt	[X]	0.04–0.10	[X]	–	v
Translating grasp	t1_xyz	part	table	[X]	0.04–0.15	[X]	–	v, e_{dyn}
Rotating grasp	t1_so3	part ($\omega \leq 1.2$ rad/s)	table	[X]	0.04–0.15	[X]	–	v, $r\theta$, e_{dyn}
Occluded grasp	t1_occ	part	occluder canopy	[X]	0.04–0.10	[X]	[X]	v, m, occ.
Moving receptacle	t2_trans	receptacle	table	[X]	0.03–0.06	[X]	–	v
Rotating receptacle	t2_se3	T-shaped receptacle	table	[X]	0.03–0.06	[X]	–	v, $r\theta$
Wire passage	t3_phase	wire loop (entry)	rotating wire	30	[X]	[X]	swept	v, $r\theta$, m
Wire passage, full	t3_full	wire loop (entry)	rotating wire	30	[X]	[X]	swept	v, $r\theta$, m
Ball capture	t4_sphere	ball	tray, ramp	[X]	[0.4–0.7]	[X]	[X]	v, e_{dyn}, m
Ball capture, table	t4_sphere_tabletop	ball	table, ramp	[X]	[0.4–0.7]	[X]	–	v, e_{dyn}
Slider capture	t4_slider	slider	table, ramp	[X]	[0.4–0.8]	[X]	–	v, e_{dyn}
Can capture	t4_can	can	table, ramp	[X]	[0.4–0.7]	[X]	–	v, $r\theta$, e_{dyn}
Post-impact grasp	t5_replan	part	striker	[X]	0.12–0.14	[X]	[X]	v, $r\theta$, e_{dyn}, m, delay

period boundaries t at which the privileged expert could make the contact of the task's first success stage. A time t belongs to it when the expert's contact frame on the target at t, taken from a rollout of the same scene with the robot held at its initial pose, (i) satisfies the geometric conditions of that stage (Table S4), (ii) has an inverse-kinematics solution within the expert's joint limits, and (iii) can be reached from the initial end-effector pose by t under the expert's per-command limits of 0.05 m per translation axis and 0.5 rad per rotation-vector component. The earliest such time is the larger of $\lceil \max_i |\Delta p_i|/0.05\,\mathrm{m} \rceil$ and $\lceil \max_i |\Delta \phi_i|/0.5\,\mathrm{rad} \rceil$ control periods, a lower bound that ignores the controller's tracking lag. Only robot–target collisions are disabled in this offline fixed-robot reference replay; target–environment collisions remain enabled, and policy rollouts retain their original physics. The window depends only on the scene and is shared by all methods. The two placement tasks start with the object already held, so their contact is the first contact of the held object with the goal receptacle, and their contact frame is the end-effector pose implied by the expert's placement target. The phase-only threading task (`t3_phase`) also starts with the wand held and succeeds without any contact, so it has no contact errors; their averages cover the other 13 tasks, and SR covers all 14. The *contact-time error* is the distance from the first-contact time to this window, and it is zero inside the window. The *position error at first contact* is the distance of the end effector from the contact frame at that time. We report both on episodes that make contact. Failures without contact stay in the SR denominator, and their contact error is missing rather than zero. *Collisions* count safety events. *Time-to-action* is the wall-clock time from the arrival of an observation to the availability of the first command (Appendix D.4), and the *action age* is the execution time minus the capture time.

Table S6. Scenario proposal ranges. Length, linear velocity, angle, and angular velocity use m, m/s, rad, and rad/s unless marked otherwise. Positions are in world coordinates except the stated ramp-local offsets. Continuous intervals are uniform proposals; discrete choices are equiprobable before feasibility checks. Evaluation replays the stored accepted scenarios, whose distribution need not remain uniform. Harmonic centers are not initial positions.

Task	Initial-state proposal	Motion or event parameters
grasp	Harmonic center $c_x \in [-.08, .08]$, $c_y \in [-.26, -.19]$, $z = .822$; yaw $[-1.1, 1.1]$.	$v_y \in [.025, .125]$, $A_x \in [.015, .035]$, $f_x \in [.3, .7]$ Hz; $\omega_z \in [-1.2, 1.2]$.
t1_belt	Target $x \in [-.08, .08]$, $y \in [-.26, -.18]$, $z = .822$; identity orientation, zero initial twist.	Belt $v_y \in [.04, .10]$.
t1_occ	Same target proposal as Belt; canopy at $(.322, -.29, 1.20)$.	Same belt range; clearance parameter .20; physical occlusion.
t1_xyz	Harmonic center $c_x \in [-.025, .025]$, $c_y \in [-.24, -.20]$, $z = .818$; yaw $[-.35, .35]$.	$v_y \in [.04, .09]$, $A_x \in [0, .03]$, $f_x \in [.25, .65]$ Hz; $\omega_z = 0$.
t1_so3	Same as XYZ.	Same translation; $\omega_z \in [-1.2, -.4] \cup [.4, 1.2]$, equal sign probabilities.
t2_trans	Goal $x \in [-.13, -.08]$, $y \in [-.18, -.12]$, $z = .825$; fixed orientation.	Goal $v_y \in [.03, .06]$; no rotation.
t2_se3	Same goal x, y range, $z = .804$; yaw $[-.3, .3]$.	Goal $v_y \in [.03, .06]$; $\omega_z \in [-.25, -.08] \cup [.08, .25]$, equal sign probabilities.
t3_phase	Wand $x, y \in [-.03, .03]$, $z = .870$, yaw $[0, 2\pi]$; wire phase $[-.25, .35]$.	Wire center $(.10, -.02, .94)$; world-X rotation $\omega_x = -\{2.5, 3, \ldots, 7.5\}^\circ$/s (11 equal-probability values).
t3_full	Wand at $(-.05, -.02, 1.04)$ with identity orientation; same wire phase range.	Same wire rotation; cradle and release requirement enabled.
t4_sphere	Ramp-local offsets: lateral $[-.015, .015]$, longitudinal/normal $(-.145, .030)$; lip $(.06, -.02, 1.10)$.	Gravity-driven from rest; gate release at tick 0; slope $[5, 15]^\circ$; target/ramp sliding friction $[.7, .9]$; unsupported capture.
t4_sphere_tabletop	Lateral $[-.015, .015]$, longitudinal/normal $(-.124, .034)$; lip $(0, -.06, .808)$.	Same slope and release; target/ramp friction $[.7, .9]$, table friction .35.
t4_slider	Lateral $[-.015, .015]$, longitudinal/normal $(-.129, .024)$; supported ramp.	Same slope and release; target/ramp friction $[.025, .045]$, table friction $[.012, .018]$.
t4_can	Lateral $[-.010, .010]$, ramp-axis yaw $[-4, 4]^\circ$; longitudinal position follows gate geometry and oriented can extent.	Same slope and release; target/ramp friction $[.25, .45]$, table friction $[.09, .15]$.
t5_replan	$x \in [-.005, .005]$, $y = -.29$, $z = .812$; yaw $[-.08, .08]$; zero initial twist.	Belt $v_y \in [.12, .14]$; equal-probability left/right striker triggered at $y = -.13$; stroke .098, pulse .35 s, actuator scale $[1, 1.1]$, force cap 4 N; command delay 50 ms.

D. Baselines, External Benchmarks, Real Robot, and Timing

D.1. Baselines

Common conditions. Table S7 summarizes the adaptations. Each baseline keeps its official architecture, inputs, action chunk, and initialization. ACT and Diffusion Policy train from scratch, $\pi_{0.5}$ is fine-tuned from `pi05_base`, and the other methods start from their released weights. All of them train on the same 11,200 demonstrations, whose commands we map to each action space through the same Cartesian controller. Dynamic-14 renders a 224×224 wrist camera in addition to the agent view, and methods whose recipes use a wrist view receive it. SE(3)-WAM and its CV-input variant use the agent view only. All methods run under the shared executor and the real-time protocol of Appendix C.3, and Table S14 adds their original executors.

Training. Each baseline follows its official recipe for the published number of updates and is trained once, with seed 0. We changed only the data loading and the action space. For DreamVLA, we ran both its original 40-epoch schedule and a 20-epoch cosine schedule and report the one that was better on development scenes. The configuration of SE(3)-WAM, in contrast, was selected by the ablations of Appendix F, a search that the baselines do not receive, and the full model, CV-input variant, and final-model core controls are each assigned three training seeds. Screening studies retain their group-specific seed counts.

Inputs. PUMA [19] receives its historical optical flow, computed on the rendered frames. DreamVLA receives seven causal RGB frames, the end-effector position and orientation, the finger positions, and a fixed task text, and it samples with ten DDIM steps and classifier-free guidance of 1.5. OpenWAM-α receives the current RGB views, joint values, gripper opening, and task text, and it generates video and actions jointly with video decoding disabled. DynamicWAM receives its flow history and kinematic descriptors.

SE(3)-augmented baselines. $\pi_{0.5}$+SE(3) and DynamicWAM+SE(3) receive the eight target-state tokens of Appendix A.1 from the same pose source, with the same validity flags and the same masking of slots without valid features. A two-layer MLP maps the tokens to the model's width. For $\pi_{0.5}$, we append them to the prefix after the image tokens, where the action expert attends to them, and for DynamicWAM they join its kinematic descriptors. The output layer of the MLP is zero-initialized, and training follows the base model's recipe. This initializes the added embeddings to zero; it does not guarantee unchanged attention or actions. These controls match target-state access, while the depth-input difference remains.

CV-input control. *SE(3)-WAM, CV input* shares the scene tokens and the action stream of SE(3)-WAM but has no forecast losses. Instead, the action stream receives constant-velocity target-pose extrapolations at the chunk's estimated execution times as input tokens. These estimates use only history and a delay estimate available at query time, never a future pose or the realized latency of that query. Training follows the same data and recipe and retains current-target RGB reconstruction. The no-future control in Table 3 has no extrapolated input tokens. Together these comparisons distinguish added trajectory inputs from future supervision; the CV-input model is a separately trained policy.

D.2. DOMINO

The external comparison uses DOMINO [19] and four methods: SE(3)-WAM, $\pi_{0.5}$, DynamicWAM, and PUMA. They share the selected single-arm Franka tasks, demonstrations, and evaluation scenes, and use the benchmark's native evaluator. The planned comparison uses the 24 prepared clean-L1 tasks, one training seed (0) per method, and 50 episodes per task and method, for 4,800 episodes. We report SR and manipulation score (MS). The task list, action semantics, and control timing are checked before formal training; test scenes are separate from interface debugging and model selection. [Complete the validated DOMINO protocol and results.] SE(3)-WAM uses domain-specific normalization and the simulator pose of the target selected by the instruction; missing target-state slots are masked as in Appendix A.1. This privileged input is disclosed separately from the RGB-based baselines. The DOMINO comparison adds no component ablations, pose-noise conditions, or additional latency sweep.

D.3. Real Robot

Platform. The robot is a Franka arm ([model]) with [gripper model] and a fixed RealSense D435 RGB-D camera, commanded through [a timestamped Cartesian interface] at [control frequency]. The camera records RGB-D at [resolution] and [frame rate], and we calibrate its extrinsics by [procedure] before each session. SE(3)-WAM uses this fixed third-person view; baselines whose recipes require a wrist view receive [per-method camera configuration]. All policies run on one RTX 5090.

Tasks and success. Table S9 defines the six tasks. T1 and T2 share a belt conveyor, T3 uses a turntable, T4 a remotely driven toy car, T5 a cue-driven collision between two balls, and T6 a ramp. Each task has its own operating range and success condition. Touching or pushing the target without completing the task, a single-finger hold, and an action after the target has left the workspace count as failures; [equipment-invalid trials and safety interventions are repeated or counted as failures by the following rule].

Trials. We compare SE(3)-WAM, $\pi_{0.5}$, DynamicWAM, and Diffusion Policy [16]. Each method runs 20 trials per task. All methods use the same pre-generated trial keys, which fix the object instance, the initial pose, and the motion parameters, sampled by [sampling protocol]. Trials run in [randomized method order], so that drift of the apparatus does not align with the methods. An operator resets the scene. A trial starts at the first usable observation, and the robot may move at any time; a trial ends at success, failure, or a timeout of [X] s. Demonstrations, development trials, and test trials come from separate sessions; test initial conditions are sampled independently under the same task protocol.

Training and adaptation. SE(3)-WAM is fine-tuned per task from the DROID base (Appendix B.5). The baselines are trained or fine-tuned on the same demonstrations, with [per-method initialization and training settings]. The marker-free tracker and offline labels share one fixed object reference (Appendix A.2); [specify the selected reference acquisition and registration settings].

Analysis. Table S10 reports successes per task, 95% binomial intervals for the pooled rates, and paired differences from a bootstrap over trial keys. The evaluation took [X] days.

D.4. Timing

We time all methods on one RTX 5090 (32 GB), the target deployment GPU of the robot. For each method, each of three

Table S7. **Baseline adaptations.** All methods train on the same 11,200 demonstrations with the inputs of their papers and run under the shared executor of Appendix C.3. Updates × batch follows the official recipe. Exec.: the execution scheme of the original paper (commands executed per chunk before replanning), used only for the original-executor control of Table S14. Params: trainable parameters, frozen encoders excluded. Seeds: planned training seeds; completion is verified before filling results.

Method	Initialization	Inputs	Updates × batch	Exec.	Params	Seeds
ACT [98]	scratch	agent + wrist RGB, robot state	50,000 × 32	temporal agg.	[X]	1
Diffusion Policy [16]	scratch	agent + wrist RGB (published horizon), robot state	50,000 × 32	8	[X]	1
$\pi_{0.5}$ [56]	`pi05_base`	current agent + wrist RGB, robot state, task text	30,000 × 256	10	[X]	1
$\pi_{0.5}$ + SE(3)	`pi05_base`	as $\pi_{0.5}$ + SE(3) motion tokens	30,000 × 256	10	[X]	1
DynamicVLA [81]	released DOM ckpt.	frames $[-2, 0]$, robot state, task text	50,000 × 44	streaming	[X]	1
PUMA [19]	official checkpoint	RGB, optical-flow history, robot state, task text	[X]×[X]	[X]	[X]	1
DreamVLA [94]	released components	seven causal agent + wrist RGB frames, robot state, task text	48,220 × 512	1	[X]	1
OpenWAM-α [75]	official foundation	current agent + wrist RGB, joints, gripper, task text	34,490 × 192	32	[X]	1
DynamicWAM [49]	official initialization	RGB, flow history, kinematic descriptors	40,000 × 128	16	[X]	1
DynamicWAM + SE(3)	official initialization	as DynamicWAM + SE(3) motion tokens	40,000 × 128	16	[X]	1
SE(3)-WAM, CV input	as SE(3)-WAM	as SE(3)-WAM, plus constant-velocity forecast tokens, no forecast losses	as SE(3)-WAM	–	[X]	3
SE(3)-WAM	frozen MAE encoder	current agent RGB-D, SE(3) and robot histories	51,740 × 512	1	523.8M	3

Table S8. **DOMINO external evaluation.** Planned comparison on 24 single-arm clean-L1 tasks: one training seed per method and 50 episodes per task and method (4,800 episodes total). SR (%) is grouped by interaction type and averaged over tasks; MS is the manipulation score. Protocol: Appendix D.2.

Method	Intercept	Track	All	MS
$\pi_{0.5}$	—	—	—	—
DynamicWAM	—	—	—	—
PUMA	—	—	—	—
SE(3)-WAM	—	—	—	—

rounds uses thirty warm-up calls and 300 measured calls over three observation sequences of eight frames, with CUDA synchronization at call boundaries and rotated method order. Each method uses the precision of its release; SE(3)-WAM uses FP32. The time-to-action includes encoding the image and generating the first command, and on the robot also tracking the newest frame. For SE(3)-WAM, we record the time of the forecast heads separately, although deployment skips them. Table 1 gives the p95 values and Table S7 the parameter counts. On the robot, traces record capture, availability, preprocessing, the start and end of inference, queue entry, command issue, and execution. The action age is the execution time minus the capture time, and we report its p50, p95, and maximum. A query misses its deadline when its time-to-action strictly exceeds the 50 ms control period; the next period then uses a remaining command or holds the current target if none is available. The fraction includes all queries, including the first query of every trial; hold-only periods do not contribute an action age. We also report peak memory (Table S11). The latency sweep of Dynamic-14 differs from wall-clock deployment, because it inserts a prescribed delay while physics advances.

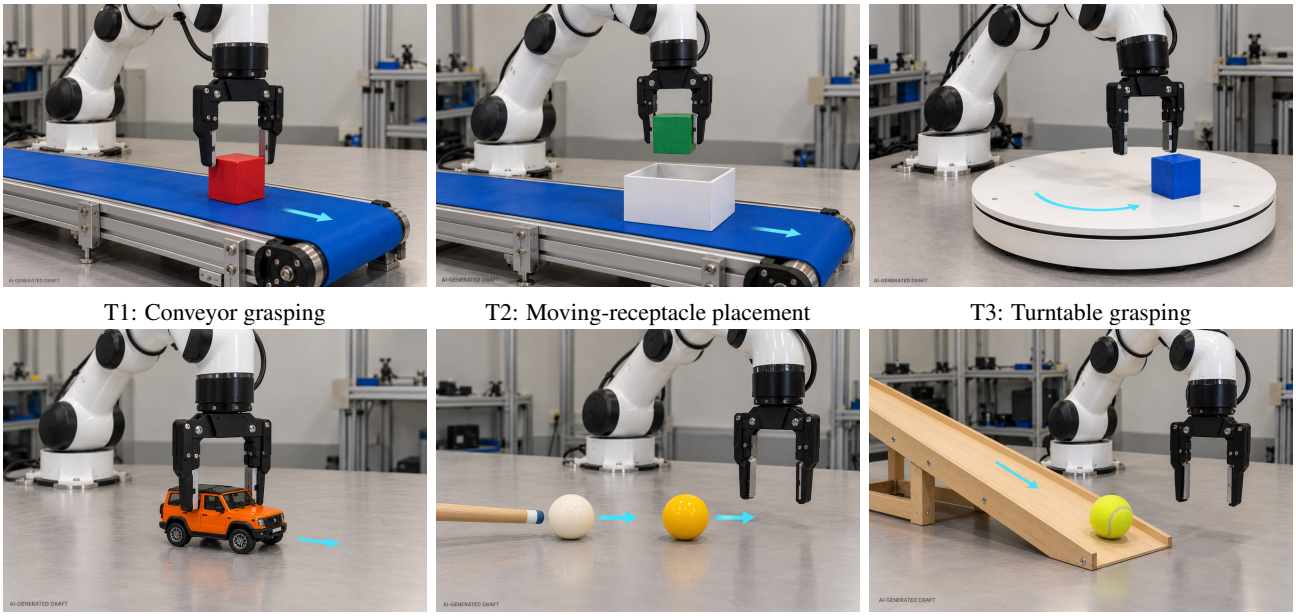


Figure S4. **Real-robot tasks.** AI-generated concept illustrations, to be replaced by photographs of the setup; arrows indicate the intended target motion. T6 shows the rolling variant; the task also includes sliding objects.

Table S9. Real-robot task definitions. $N = 100$: training demonstrations per task; 20 additional demonstrations per task are reserved for validation.

Task	Device, motion, and objects	Operating range; N	Success and failure criteria
T1	Conveyor transports a graspable cube.	[belt speed]; 100	[stable grasp and lift; failures]
T2	Conveyor transports an open box; the arm places a cube into it.	[belt speed]; 100	[placement and retention; failures]
T3	Turntable carries a cube around its axis.	[angular speed, radius]; 100	[stable grasp and lift; failures]
T4	Remotely driven toy car moves on the tabletop.	[speed, path]; 100	[car grasp and lift; failures]
T5	A cue-driven ball strikes a second ball; the arm intercepts the second ball after the collision.	[impact and outgoing speed]; 100	[post-impact capture; failures]
T6	A ball rolls or a block slides down a ramp toward the interception region.	[slope, release, speed]; 100	[capture and retention; failures]

Table S10. **Real-robot successes** per task (successes/trials, 20 trials per task; tasks in Table S9). All: pooled success rate (%) with its 95% binomial interval. Δ: paired difference from SE(3)-WAM (points) with a bootstrap interval over trial keys.

Method	T1	T2	T3	T4	T5	T6	All	Δ
$\pi_{0.5}$	—	—	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—	—	—
Diffusion Policy	—	—	—	—	—	—	—	—
SE(3)-WAM	—	—	—	—	—	—	—	–

Table S11. **Latency breakdown** of one query of SE(3)-WAM on one RTX 5090 at batch size 1 (ms, p50 / p95), and action age on the robot.

Stage	p50	p95
Pose tracking (per frame, robot)	—	—
Image encoding	—	—
World stream (once per query)	—	—
Action stream (5 Euler steps)	—	—
Time-to-action (total)	—	—
Forecast heads (training only, for reference)	—	—
Action age at execution (robot)	—	—
Queries over 50 ms (robot, %)	—	

E. Additional Results

E.1. Per-Task Results and Outcomes

Table S12 breaks the held-out Dynamic-14 results of Table 1 down by task for every method, with the paired interval of SE(3)-WAM against the strongest other method on each task. Table S13 reports how episodes end. A method that trades one failure for another, e.g., fewer collisions for more timeouts, shows this only here. Every failure stays in the success-rate denominator.

E.2. Protocols and Latency

Table S14 evaluates every method under three further protocols. Paused physics removes the cost of computation. Equal latency charges every method the p95 latency of SE(3)-WAM and so separates a method's speed from the way it uses time. The original executors test whether our shared executor puts any method at a disadvantage. With paused physics, the difference between SE(3)-WAM and [DreamVLA] is [$\pm$X.X] points, and under equal latency it is [$\pm$X.X] points. The original executors change the SR of the baselines by at most [X.X] points. Table S15 gives success at every added latency of the sweep, and its rows for SE(3)-WAM and the strongest baseline are the data of Fig. 1 (right).

E.3. Negative and Null Results

Screening results. The history, forecast, encoder, and coupling comparisons use the screening recipes of Appendix F. Their original splits and seed counts must accompany each result. Q3 coupling effects changed sign across seeds; this does not establish final-scale equivalence between training-only supervision and online forecast feedback. The new four-control study reports signed differences and paired intervals in Table 3. [Summarize completed null and negative effects at their actual scale.]

The completed history and forecast screening results are reported at their original scale in Tables S17 and S20. Table S18 reports the stationary-history intervention on the frozen screening checkpoint; it is not a final-scale training ablation.

Unsolved tasks. The SE(3) placement into the rotating receptacle (t2_se3) fails for every method (Table S12). [The policies reach the receptacle but miss its angular tolerance before it turns away.] No variant of SE(3)-WAM solves the full wire passage (t3_full), which adds grasping and releasing the wand to the passage, whereas [OpenWAM-α] succeeds in [XX]% of these episodes. On the passage that starts with the wand grasped (t3_phase), the success of SE(3)-WAM ranges from [0] to [71]% across training seeds.

Other failures. The collapse of Fig. S6a is weakest for the capture tasks (Spearman $\rho = [X.XX]$), where success also depends on when the gripper closes. The three synthetic-noise conditions change SR by [report the three signed differences from the clean reference]; these results are separate from physical-tracker calibration. Most remaining failures of SE(3)-WAM are [e.g., timeouts after a missed first grasp] (Table S13).

E.4. Forecasts at Execution Time

Fig. S5 aligns the query, the forecast, the ground truth, the issued actions, and the first contact on physical timestamps. The examples show where forecast errors arise—e.g., at the moment of an impact or when a rolling object leaves its ramp—alongside contact-time and pose errors. These aligned diagnostics do not establish that the deployed action stream reads the decoded forecasts or that a forecast error caused a particular action error.

E.5. Supplementary Video

The supplementary video shows real-robot trials of SE(3)-WAM and the strongest baseline on all six tasks with the tracked poses overlaid, a set of Dynamic-14 episodes per task family with the forecasts of the training-time heads, and the failure cases discussed in Sec. 4.3.

Table S12. **Per-task held-out success** (%) on Dynamic-14 for every method of Table 1, mean of three training seeds for our variants and one for baselines, real-time protocol. DP: Diffusion Policy. DVLA: DynamicVLA. DrVLA: DreamVLA. OWAM: OpenWAM-α. DWAM: DynamicWAM. +: with our SE(3) history tokens. CV: SE(3)-WAM with CV input. σ: SE(3)-WAM with nominal synthetic temporally correlated pose noise under the approved robustness setting, reusing those episodes. Δ: SE(3)-WAM minus the strongest other method on that task, with its 95% paired interval. Task codes are listed in Table S5.

Task	ACT	DP	$\pi_{0.5}$	$\pi_{0.5}$+	DVLA	PUMA	DrVLA	OWAM	DWAM	DWAM+	CV	σ	**Ours**	Δ [95% CI]
Harmonic grasp	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Conveyor grasp	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Translating grasp	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Rotating grasp	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Occluded grasp	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Moving receptacle	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Rotating receptacle	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Wire passage	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Wire passage, full	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Ball capture	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Ball capture, table	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Slider capture	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Can capture	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Post-impact grasp	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Task-balanced	—	—	—	—	—	—	—	—	—	—	—	—	—	—

Figure placeholder: Fig. S5 — forecasts aligned with what happened

Rows 1–4: the median-error held-out episode of each motion type (conveyor, rotating target, rolling capture, post-impact); **row 5:** a failure. **Columns:** the query frame at τ_t; the frame at the first command's execution time τ^{a}; and a 3D inset in the base frame. **Overlays:** blue past SE(3) poses; orange predicted target triads at $\tau_t + \delta_h$ and sparse flow arrows, read from the training-time heads; gray ground-truth poses at the same physical times; purple issued gripper path; a marker at first contact. A strip under each row plots translation and rotation error vs. horizon for learned SE(3), constant velocity, and constant acceleration. Annotate the failure cause of row 5 in red.

Figure S5. Forecasts of the training-time heads on held-out episodes, aligned in physical time with the ground truth and the issued actions. Rows 1–4 show the median-error episode of each motion type, and row 5 shows a failure.

Table S13. **Outcome categories** on held-out Dynamic-14 scenes (% of episodes, seed-averaged, real-time protocol). Categories follow Appendix C.1, and rows sum to 100. The noisy-pose row reuses the nominal synthetic correlated-noise episodes from the robustness study.

Method	Success	Safety	Task	Bound.	Timeout	Inval.
ACT	—	—	—	—	—	—
Diffusion Policy	—	—	—	—	—	—
$\pi_{0.5}$	—	—	—	—	—	—
$\pi_{0.5}$ + SE(3)	—	—	—	—	—	—
DynamicVLA	—	—	—	—	—	—
PUMA	—	—	—	—	—	—
DreamVLA	—	—	—	—	—	—
OpenWAM-α	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—
DynamicWAM + SE(3)	—	—	—	—	—	—
SE(3)-WAM, CV input	—	—	—	—	—	—
SE(3)-WAM, noisy poses	—	—	—	—	—	—
SE(3)-WAM	—	—	—	—	—	—

Table S14. **Protocol controls** on held-out Dynamic-14 scenes (SR, %). Paused: shared executor, with physics stopped during inference. Equal latency: shared executor, with every method charged the p95 latency of SE(3)-WAM. Original executor: real time with each method's own execution scheme. Real time: the protocol of Table 1. Late: queries in real time whose time-to-action exceeds the 50 ms control period (%).

Method	Paused	Equal latency	Original executor	Real time	Late (%)
ACT	—	—	—	—	—
Diffusion Policy	—	—	—	—	—
$\pi_{0.5}$	—	—	—	—	—
$\pi_{0.5}$ + SE(3)	—	—	—	—	—
DynamicVLA	—	—	—	—	—
PUMA	—	—	—	—	—
DreamVLA	—	—	—	—	—
OpenWAM-α	—	—	—	—	—
DynamicWAM	—	—	—	—	—
DynamicWAM + SE(3)	—	—	—	—	—
SE(3)-WAM, CV input	—	—	–	—	—
SE(3)-WAM	—	—	–	—	—

Table S15. **Success versus added latency** on held-out Dynamic-14 scenes (SR in %, real-time protocol). The 0 and 150 ms columns equal the All and +150 ms columns of Table 1. Slope: least-squares change of SR per 100 ms of added latency (points).

	Added latency (ms)					
Method	0	50	100	150	200	Slope
ACT	—	—	—	—	—	—
Diffusion Policy	—	—	—	—	—	—
$\pi_{0.5}$	—	—	—	—	—	—
$\pi_{0.5}$ + SE(3)	—	—	—	—	—	—
DynamicVLA	—	—	—	—	—	—
PUMA	—	—	—	—	—	—
DreamVLA	—	—	—	—	—	—
OpenWAM-α	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—
DynamicWAM + SE(3)	—	—	—	—	—	—
SE(3)-WAM, CV input	—	—	—	—	—	—
SE(3)-WAM	—	—	—	—	—	—

F. Core Ablations and Screening Studies

Final-scale confirmation. Table 3 contains four independently trained controls of the 24-layer final model, with three training seeds each. They keep the final data, initialization recipe, training exposure, and other components fixed (Appendix A.6). Evaluation uses 1,400 held-out scenes under the real-time protocol at 0 and 150 ms added delay, with shared scene keys and inference seeds. The full-model reference is reused from the main evaluation. The four controls are:

- **No target history:** retain the current image, current target pose, and eight robot states; remove past target poses and their velocity and acceleration features.
- **No future supervision:** disable SE(3), scene-flow and visibility, future depth, DINO, and dynamic-RGB losses; retain current-target RGB reconstruction and the action loss.
- **No scene-flow supervision:** disable only scene-flow and visibility losses; retain dedicated SE(3) and all appearance targets.
- **No dedicated SE(3):** disable only the target-pose forecast loss; retain all appearance targets and the same 14^2 scene-flow queries over the target, robot, and surroundings.

The last control assigns the target the same coarse flow representation as the rest of the scene; it introduces no new target-track head. Since scene flow includes the target and robot, removing it does not isolate environment information. Results are pending: [complete the four controls and their paired intervals before drawing final-scale conclusions].

Screening scope. The broader studies used 4 layers of width 768 and 4 epochs, generally 40,104 updates at global batch size 16. HISTORY1 used a frozen DINO-L encoder and one training seed, with a first-epoch cosine schedule followed by a low learning rate. FUTURE0 used the same depth and width and one seed, but a cosine schedule over all four epochs. Q3-INFO0 used MAE and three seeds for each of R0–R5; BACKBONE0 compared its two backbones with three seeds. The added AUX0 appearance conditions used one seed. These groups are not one matched final-model comparison. Development results motivate design choices; they retain their original split and recipe when cited. The retained screening tables describe completed conditions at their original scale; they do not add 24-layer training arms to the core confirmation.

F.1. Shared Components

The encoder and backbone comparisons belong to the screening scale. They compare a trained CNN, frozen DINOv3 ViT-S/16 and ViT-L/16 [63], and MAE ViT-B/16 [26], with their original group-specific recipes and seed counts. The MAE reference and the BACKBONE0 comparison have three seeds; this does not make every encoder or auxiliary-target condition a three-seed study. The final model uses MAE; DINO-L remains a future feature target. Table S16 retains the screening comparison layout.

Table S16. **Observation-encoder screening layout.** The 4-layer studies retain their source recipes and seed counts (Appendix F.1); the MAE reference has three seeds. Blank cells are unfilled measurements, not a promise of final-scale retraining. Params: encoder parameters; p95: time-to-action.

Encoder	Params	Trained	SR	SR_{150}	p95
CNN	—	yes	—	—	—
DINOv3 ViT-S/16	22M	frozen	—	—	—
DINOv3 ViT-L/16	300M	frozen	—	—	—
MAE ViT-B/16*	86M	frozen	—	—	—

Table S17. **History screening.** Four layers, width 768, frozen DINO-L, one training seed, and the original synchronous evaluation on 1,400 scenes. The first eight conditions receive the current target pose; the final condition adds target-motion history. These are not final-scale real-time results.

History input	Success / 1,400	SR (%)
Current image	748	53.43
RGB window	662	47.29
Optical flow	720	51.43
Flow + motion statistics	723	51.64
Codec motion vectors	733	52.36
Video-VAE latent	710	50.71
Compressed window (age)	634	45.29
Compressed window (uniform)	670	47.86
Current image + target-motion history	824	58.86

F.2. History

HISTORY1 screened current-state input, target motion history, RGB windows, Farnebäck flow [20], flow with motion statistics [49], codec motion vectors, video-VAE latents, and compressed windows. Its nine trained conditions use one seed each. Table S17 reports these nine completed conditions. Only the no-target-history control is repeated at the final scale in the core confirmation.

Stationary-history intervention. The completed intervention in Table S18 uses the frozen 4-layer HISTORY1 checkpoint, one training seed, and the original synchronous executor. It replaces past target poses by the current pose before recomputing kinematics; current pose, image, robot state, timestamps, and validity are retained. The original history result is reused. This intervention changes the input distribution and does not replace the separately trained final-scale no-history control.

Table S18. **Stationary-history intervention.** One frozen 4-layer HISTORY1 checkpoint, training seed 0, and its original synchronous protocol. Past target poses are replaced by the current pose and kinematics recomputed; the unchanged-history result is reused. These 1,400 original scenes are separate from the final real-time confirmation.

Target history	Success / 1,400	SR (%)
Unchanged	824	58.86
Stationary replacement	9	0.64

Pose noise. The separate robustness evaluation freezes the model and reuses the real-time clean reference. It adds three conditions: nominal synthetic temporally correlated pose noise, twice its amplitude, and nominal noise with contiguous missing observations. The perturbation applies to current and historical estimates before history caching and kinematics. Actual tracker calibration and closed-loop tracking remain pending (Table S1). No history-only noise crossover or additional visual-history training is required for this three-condition evaluation. The nominal process combines a fixed per-episode bias and a stationary correlated component, each with per-axis standard deviations of 5 mm and 1° for translation and rotation. The correlation time is 200 ms at 50 ms sampling; the doubled condition scales both components. Dropout lasts four ticks (200 ms), with geometrically distributed valid gaps of mean 76 ticks, giving a long-run missing fraction of 5%. These parameters are set before evaluation. All three final-model training seeds use the same matched scenes at zero added latency; three noise conditions are not three training seeds.

F.3. Forecast

FUTURE0 screened fifteen target configurations with one seed each. Table S19 defines their representations, and Table S20 reports the completed conditions. P09 uses target 2D tracks; it is distinct from the final-scale uniform 3D-flow control. The table also reports AUX0's four MAE appearance-target conditions and their existing all-target reference, all at seed 1. These use a different encoder and recipe from FUTURE0; their scores are compared within each group. They cannot be pooled with Q3's three-seed coupling results or presented as final-scale evidence that geometry outperforms appearance.

Role coverage. The core no-SE(3) control retains the original 14^2 scene flow, and the no-flow control retains SE(3) and all appearance targets. Since the flow covers target, robot, and context, the latter tests scene-flow supervision as a whole. The study makes no claim that a coarse grid matches denser grids.

Extrapolation and forecast errors. Constant-velocity and constant-acceleration estimates from causal pose history can be compared offline with learned forecasts at the logged execution times. Table S21 compares their position errors on the same held-out observations at a fixed 400 ms horizon. CV and CA use only causal pose history; the learned head reads the frozen full-model states. These are offline prediction diagnostics and introduce no controller or training arm. They do not show that the deployed policy reads decoded forecasts. The separately trained CV-input policy remains in Table 1; no additional CV/CA-input training combinations are added.

F.4. Use of the Forecast and Backbone

BACKBONE0 compared shared attention with stream-specific feed-forward networks against separate world/action parameter groups at about 47.3M backbone parameters. Its three-seed results belong to this 4-layer setting.

Q3-INFO0 trained R0–R5 at the same screening scale with three seeds each (Appendix A.5). Its R1 contains world queries with no edge to the actions, whereas the final model reads out from summary tokens without these queries. The Q3 comparisons therefore do not directly measure the final model. Coupling effects changed sign across seeds; this supports the choice of a simpler deployment but does not establish statistical equivalence. R1 minus R0 was [3.14] percentage points on average, with seed-wise differences of [−1.29], [+9.64], and [+1.07]. The core confirmation does not repeat R5 or the other coupling designs at 24 layers.

Deployment check. On the final checkpoint, running unused forecast heads can check equality of the action output for fixed inputs and flow-matching noise and quantify the added computation. Since the real-time executor advances physics during this computation, equal outputs at fixed inputs do not imply equal closed-loop success. No forecast-swap, time-shift, or oracle-input matrix is added for R2/R5.

Table S19. Future-target definitions for the completed screening conditions. Shape excludes batch and query axes. Displacements are cumulative from the current state, except for codec vectors, which keep the correspondence of the decoder. All targets come from the recorded demonstrations. Future depth, DINO, and dynamic RGB are also the training-only auxiliary targets of the selected model.

Target	Shape	Support, units, and validity
Target pose	8×6	Cumulative translation (m) and left-relative rotation vector (rad). Requires a valid pose and query.
Scene flow, 14×14 grid	$196 \times 8 \times 3$	Fixed source grid, cumulative displacement in base-frame meters. Requires valid source depth and a finite future displacement.
Target tracks (2D)	$32 \times 8 \times 3$	Target-mask source points with two image-displacement channels and visibility; padded points are invalid. This is the P09 target, not a 3D rigid-pose fit.
Object DINO	8×1024	Mask-weighted pooling of future patch features with channel layer normalization. Requires a positive mask area.
Scene tracks (2D)	$196 \times 8 \times 3$	Grid source points with two image displacements divided by 224 and a visibility logit.
Dense 2D displacement	$8 \times 2 \times 224 \times 224$	Fixed source pixels, displacement divided by 224. Requires a valid source and a finite projection.
Codec vectors	$8 \times 2 \times 64 \times 64$	Each future/current pair is encoded as P/I frames, and vectors are divided by 64. Only valid vector regions count.
Video-VAE latent	$48 \times 2 \times 14 \times 14$	Frozen Wan2.2 VAE posterior mean with the VAE's own normalization. The two groups end at 200 and 400 ms.
Future depth	$8 \times 1 \times 224 \times 224$	Metric depth on finite positive pixels.
Future DINO	$8 \times 196 \times 1024$	Frozen DINOv3 ViT-L/16 teacher [63], 14×14 patches, channel layer normalization.
Dynamic RGB	$8 \times 3 \times 224 \times 224$	RGB divided by 255 on surfaces moving more than 1 mm, including the robot.

Table S20. **Completed forecast screening.** Both groups use 4 layers of width 768 and 4 epochs, with their original synchronous evaluation. FUTURE0 uses frozen DINO-L and seed 0; its 140 development scenes and disjoint 1,260-scene complement remain separate. AUX0 uses frozen MAE-B, seed 1, and 1,400 original scenes; it retains scene flow, SE(3), and current-target RGB reconstruction. The all-auxiliary MAE reference is reused. Compare conditions within each recipe, not across groups.

ID	Future supervision	Dev. success / 140	Success / 1,260	SR_{1260} (%)
P00	None	88	791	62.78
P01	3D scene flow	95	782	62.06
P02	SE(3)	92	789	62.62
P03	Scene flow + SE(3)	93	786	62.38
P04	Video-VAE latent + SE(3)	92	783	62.14
P05	Dense 2D displacement + SE(3)	87	746	59.21
P06	Codec motion + SE(3)	86	795	63.10
P07	Sparse scene 2D tracks + SE(3)	90	807	64.05
P08	Scene flow + object DINO	89	794	63.02
P09	Scene flow + target 2D tracks	88	808	64.13
A_α	P03 + motion magnitude	89	778	61.75
P12	P03 + depth	90	791	62.78
P13	P03 + DINO	92	788	62.54
P14	P03 + dynamic RGB	89	790	62.70
P15	P03 + depth / DINO / dynamic RGB	94	802	63.65
AUX0 MAE-B: future appearance auxiliaries		Success / 1,400		SR_{1400} (%)
None		894		63.86
Depth only		867		61.93
DINO only		946		67.57
Dynamic RGB only		926		66.14
All three (existing reference)		995		71.07

Table S21. **Offline prediction errors.** Target-position error at 400 ms (mm) on the same held-out observations. CV and CA are analytic extrapolations from causal pose history; the learned head reads frozen full-model states. These diagnostics are distinct from the trained CV-input policy in Table 1.

	Const. vel.	Harmonic	Roll/slide	Impact
CV extrapolation	—	—	—	—
CA extrapolation	—	—	—	—
Learned pose head	—	—	—	—

Table S22. **Coupling at the screening scale.** Q3-INFO0 used three seeds per condition at 4 layers and width 768. R1 has world queries and is distinct from the final model. SR and p95 retain the source evaluation protocol and hardware; blank cells are unfilled.

Coupling	SR mean	SR std.	p95 (ms)
R0 (future losses off)	—	—	—
R1 (queries, no edge)	—	—	—
R2 (world→action)	—	—	—
R3 (action→world)	—	—	—
R4 (bidirectional)	—	—	—
R5 (decoded geometry)	—	—	—

G. Error Analysis of Acting Under Delay

This appendix derives the error decomposition of Sec. 3.1, extends it to rotation, the context, and chunked execution, derives simple scaling relations, gives magnitudes for Dynamic-14, and states where it does not apply. The decomposition motivates the design of SE(3)-WAM and is not a model of success.

G.1. Translation

Let the true target position at the execution time be $\mathbf{p}(\tau^{\mathrm{a}}) = \mathbf{p}(\tau_t) + \mathbf{v}\Delta + \boldsymbol{\rho}$ and the policy's estimate $\hat{\mathbf{p}}(\tau^{\mathrm{a}}) = \hat{\mathbf{p}}(\tau_t) + \hat{\mathbf{v}}\hat{\Delta} + \hat{\boldsymbol{\rho}}$. Subtracting and adding $\mathbf{v}\hat{\Delta}$ gives the identity

$$\begin{aligned}\hat{\mathbf{p}}(\tau^{\mathrm{a}}) - \mathbf{p}(\tau^{\mathrm{a}}) = \big[\hat{\mathbf{p}}(\tau_t) - \mathbf{p}(\tau_t)\big] + (\hat{\mathbf{v}} - \mathbf{v})\hat{\Delta} \\ + \mathbf{v}(\hat{\Delta} - \Delta) + (\hat{\boldsymbol{\rho}} - \boldsymbol{\rho}),\end{aligned} \tag{S4}$$

and the triangle inequality yields Eq. 3. Since $\mathbf{v}$ is the velocity at τ_t, $\boldsymbol{\rho}$ is exactly the part of the motion that constant-velocity extrapolation misses. Under a constant acceleration $\mathbf{a}$, $\boldsymbol{\rho} = \frac{1}{2}\mathbf{a}\Delta^2$, and an acceleration estimate from the history removes part of e_{dyn}. Impacts, rolling contact, and friction changes leave a remainder that only a model of the dynamics can reduce. For a learned predictor, the split of its estimate into $\hat{\mathbf{v}}\hat{\Delta}$ and $\hat{\boldsymbol{\rho}}$ is not identified, because any predicted position can be written in this form. We therefore use the decomposition to say which information each term requires. Our experiments do not separate the terms inside a learned predictor. Instead, they vary the lead, the speed, the pose noise, and the timing, which act on known terms, and they compare variants that differ in the information they receive (Appendix G.5).

G.2. Rotation and the Contact Point

We write $R(\tau^{\mathrm{a}}) = \mathrm{Exp}(\boldsymbol{\omega}\Delta + \boldsymbol{\eta})R(\tau_t)$ and $\hat{R}(\tau^{\mathrm{a}}) = \mathrm{Exp}(\hat{\boldsymbol{\omega}}\hat{\Delta} + \hat{\boldsymbol{\eta}})\hat{R}(\tau_t)$. When R and $\hat{R}$ rotate about a common fixed axis, the exponentials compose additively, and

$$\theta = \big\| \mathrm{Log}(\hat{R}(\tau^{\mathrm{a}})R(\tau^{\mathrm{a}})^{\top})^{\vee} \big\| \le \theta_{\mathrm{loc}} + \hat{\Delta}\, e_{\omega} + \|\boldsymbol{\omega}\|\, e_{\tau} + e_{\eta}, \tag{S5}$$

where θ_{loc} is the current orientation error and e_{ω} and e_{η} are defined as for translation. For general rotations, the Baker–Campbell–Hausdorff formula adds terms of second order in the angles. A contact point at $\mathbf{c}$ in the object frame, with $\|\mathbf{c}\| = r$, has the world position $\mathbf{p} + R\mathbf{c}$, so its error is $(\hat{\mathbf{p}} - \mathbf{p}) + (\hat{R}R^{\top} - I)R\mathbf{c}$. The operator norm of $\hat{R}R^{\top} - I$ for a rotation by θ is $2\sin(\theta/2)$, which gives

$$e_{\mathrm{c}} \le \|\hat{\mathbf{p}} - \mathbf{p}\| + 2r\sin(\theta/2) \le \|\hat{\mathbf{p}} - \mathbf{p}\| + r\theta, \tag{S6}$$

which is the contact bound of Sec. 3.1. Orientation errors therefore enter the contact geometry scaled by the lever arm, and a grasp or placement that depends on orientation needs an SE(3) target state. The planned real-ball protocol uses ball-center translation and excludes target rotation and angular kinematics from inputs, rotation supervision, and rotation errors (Appendix A.2). This task-specific choice leaves robot rotation commands and simulation SE(3) states unchanged; it is not a general symmetry-handling result.

G.3. Context and Clearance

Let $\mathcal{P}$ be the set of context points and times that the executed path passes, and suppose that the path keeps every point $\mathbf{x}$ of the robot and the carried object at least m away from the *predicted* context positions $\hat{\mathbf{y}}_t(\mathbf{z}, \tau)$ for $(\mathbf{z}, \tau) \in \mathcal{P}$. A collision requires $\mathbf{x} = \mathbf{y}_t(\mathbf{z}, \tau)$ for some $(\mathbf{z}, \tau) \in \mathcal{P}$, and then $\|\hat{\mathbf{y}}_t(\mathbf{z}, \tau) - \mathbf{y}_t(\mathbf{z}, \tau)\| = \|\hat{\mathbf{y}}_t(\mathbf{z}, \tau) - \mathbf{x}\| \ge m$, so a collision requires a prediction error of at least m somewhere along the path. Context precision finer than m thus has no first-order effect on success. The bound concerns the size of the error and says nothing about sampling. A flow sampled on a grid misses structures thinner than its spacing and errs at motion boundaries. Both effects enter the path error e_F, which cannot be inferred from grid spacing alone. The margin is itself a design variable, since a policy can keep a larger margin when its context estimate is coarse, at some cost in reachability or time.

G.4. Leads Under Rolling Replanning

Under the real-time executor of Appendix C.3, query t has total latency ℓ_t, including any added delay. Its chunk is adopted after $k_t\Delta_a$, with $k_t = \max(1, \lceil \ell_t/\Delta_a \rceil)$, and the next query starts at that same boundary. For chunk length $K = 8$, commands that reach execution have leads

$\Delta_{t,j} = (k_t + j)\Delta_a + d_{\text{task}}$, $j = 0, \ldots, \min(K, k_{t+1}) - 1$, before replacement or exhaustion. Here d_{task} is the built-in command delay: 50 ms for post-impact grasp and zero otherwise. Holding after exhaustion adds no new command. With $k_t = k_{t+1} = k \leq 4$ (total latency at most 200 ms), the largest lead is $(2k-1)\Delta_a + d_{\text{task}}$ and remains inside the 400 ms forecast, including the task delay. At larger total delays, the last commands can execute beyond this horizon. The full model's action stream does not receive the latency, so it does not adapt its commands to a particular delay. Replanning at the first eligible boundary keeps the leads short; the latency sweep measures the effect of longer leads on success.

G.5. Scaling Relations

The decomposition gives the following reference relations. Fig. S6 reports motion and synthetic-noise diagnostics.

Critical lead. A policy that does not compensate motion ($\hat{\mathbf{v}} = 0$, $\hat{\boldsymbol{\rho}} = 0$) incurs $\|\mathbf{v}\|\Delta$ in addition to e_{loc} and $\|\boldsymbol{\rho}\|$. With a small localization error and smooth motion, its contacts fail once $\|\mathbf{v}\|\Delta > \varepsilon$, i.e., beyond the critical lead $\Delta^\star = \varepsilon/\|\mathbf{v}\|$, and its success depends on the task, the speed, and the latency only through $\|\mathbf{v}\|\Delta/\varepsilon$. Table S5 lists ε, $\|\mathbf{v}\|$, and $\Delta^\star$ for every task. Fig. S6a plots the success of each method against $\|\mathbf{v}\|\Delta/\varepsilon$, computed per episode at the first command's execution time and pooled over the existing task and latency conditions, together with the Spearman correlation and a fitted logistic curve for the reactive policies. The reactive policies [fall on one curve] (Spearman $\rho = [X.XX]$), and their success halves at [X.X]. The decomposition gives only a threshold of about one, not the shape of this curve. The capture tasks scatter most around it, because their success also depends on when the gripper closes.

Pose noise. A least-squares velocity estimate from n equally spaced poses that span T, each with independent noise σ per axis, has the standard deviation $\sigma\sqrt{12(n-1)/(n(n+1))}/T \approx \sigma\sqrt{12/n}/T$ [5]. A pose history therefore keeps $\hat{\Delta}e_v$ below the tolerance up to $\sigma^\star = \varepsilon T\sqrt{n}/(\Delta\sqrt{12})$. For $\varepsilon = 10$ mm, $T = 350$ ms, $n = 8$, and $\Delta = 200$ ms, this gives $\sigma^\star \approx 14$ mm. A learned estimator with a motion prior can do better, so $\sigma^\star$ is the value for a model-free linear estimate under independent noise. Our robustness test instead uses three synthetic conditions: temporally correlated noise, twice its amplitude, and nominal noise with contiguous missing observations. It reuses the clean real-time reference and does not estimate a noise crossover or claim calibration to a physical tracker.

Clearance. A context forecast with path error e_F avoids collisions while $m > e_F$, under the assumptions of Appendix G.3. This is a geometric sufficient condition, not an empirical equivalence claim between coarse and fine flow grids.

Figure placeholder: Fig. S6 — uncompensated motion and pose noise

(a) x: $\|\mathbf{v}\|\Delta/\varepsilon$ at the first command's execution time (log); y: SR. One point per method × family × latency, reusing the main latency sweep; reactive policies (gray) with a fitted logistic curve and Spearman ρ, the threshold at 1 marked; motion-compensating baselines and SE(3)-WAM (orange). **(b)** x: clean reference, nominal correlated noise, twice the amplitude, and nominal noise with contiguous missing observations; y: SR. Frozen weights and the same real-time protocol; paired intervals from matched scenes.

Figure S6. **Motion and pose-noise diagnostics.** (a) Success against uncompensated motion relative to the tolerance, comparing reactive and motion-compensating methods. (b) Robustness under the three synthetic pose-noise conditions, relative to the clean reference.

G.6. Magnitudes in Dynamic-14

Table S23 evaluates the terms for representative Dynamic-14 settings (Table S6). Each of them is comparable to or larger than the millimeter-scale tolerances of the success conditions (Table S4).

Table S23. Illustrative magnitudes of the terms of Eq. 3 for a 200 ms lead. The values follow from the stated quantities and are not measurements.

Term	Setting	Size
$\|\mathbf{v}\|\Delta$	conveyor at 0.10 m/s	20 mm
$\hat{\Delta}e_v$	10% speed misestimate	2.8 mm
$\|\mathbf{v}\|e_\tau$	50 ms timing error (one period)	7.0 mm
$2r\sin(\theta/2)$	$\omega = 1.2$ rad/s unpredicted, $r = 3$ cm	7.2 mm
e_{dyn}	ball on a 10° ramp, no acceleration model	24 mm
ε	grasp of the conveyor part	[X] mm
grid spacing	14×14 flow at the workspace	[X] mm

The ramp entry uses the acceleration of a solid sphere that rolls without slipping, $a = \frac{5}{7}g\sin\alpha \approx 1.2\,\text{m/s}^2$ at $\alpha = 10°$, and $\frac{1}{2}a\Delta^2$ for $\Delta = 0.2$ s.

G.7. Scope

The decomposition addresses the approach to a rigid target whose motion is exogenous until contact. It does not cover deformable or articulated targets, whose motion field on $\mathcal{O}$ is not determined by six numbers. It also does not cover the motion after contact, which depends on the action itself and calls for action-conditioned prediction, the case in which letting the world read the action could matter. Scenes in which the target's role changes over time (e.g., sequential multi-object tasks) are outside its scope as well. Perception

quality enters only through e_{loc} and e_v, and we measure it separately (Table S1). The bound is not a probability of success. It identifies which errors scale with which factors.